The Concise TypeScript Book

Simone Poggiali

The Concise TypeScript Book

The Concise TypeScript Book ger en omfattande och kortfattad översikt över TypeScripts möjligheter. Den erbjuder tydliga förklaringar som täcker alla aspekter av den senaste versionen av språket, från dess kraftfulla typsystem till avancerade funktioner.

Oavsett om du är nybörjare eller erfaren utvecklare är den här boken en ovärderlig resurs för att förbättra din förståelse för och dina färdigheter i TypeScript.

Den här boken är helt gratis och har öppen källkod.

Jag anser att teknisk utbildning av hög kvalitet ska vara tillgänglig för alla. Därför håller jag boken fritt tillgänglig och uppdaterar den regelbundet med förbättringar och nya exempel.

Upptäck **The Concise TypeScript Book Plus Edition**.



För läsare som vill gå längre än utgåvan med öppen källkod innehåller **The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7** ytterligare och exklusivt innehåll med fokus på praktisk tillämpning.

Plus-utgåvan innehåller:

- **Uppdaterad för TypeScript 7** — täcker de senaste funktionerna och språkförbättringarna i TypeScript 7.
- **TypeScript med React** — praktisk vägledning för typning av komponenter, props, hooks, händelser, children, refs och vanliga React-mönster.

- **TypeScript-recept för verkliga projekt** — fokuserade exempel som tar upp praktiska problem som utvecklare stöter på när de bygger och underhåller TypeScript-applikationer.

Genom att köpa Plus-utgåvan stöder du också direkt den fortsatta utvecklingen och underhållet av den kostnadsfria boken med öppen källkod.

Plus-utgåvan finns på engelska och italienska på Amazon världen över. Utforska Plus-utgåvan och köp den på Amazon.

Stöd projektet

Om den kostnadsfria boken hjälpte dig att lösa en bugg, förstå ett svårt begrepp eller komma vidare i karriären kan du överväga att stödja mitt arbete genom att betala valfritt belopp, med ett rekommenderat bidrag på **$5**, eller bjuda på en kopp kaffe.

Ditt stöd hjälper mig att hålla innehållet uppdaterat och utöka det med nya exempel, tydligare förklaringar och ytterligare praktisk vägledning.





Översättningar

Denna bok har översatts till flera språkversioner, inklusive:

Engelska

Bulgariska

Tyska

Franska

Indonesiska

Italienska

Japanska

Koreanska

Polska

Portugisiska (Brasilien)

Svenska

Turkiska

Vietnamese

Kinesiska

Spanska

Thailändska

Ryska

Arabiska Tjeckiska

Nedladdningar och webbplats

Du kan också ladda ner EPUB-versionen:

https://github.com/gibbok/typescript-book/tree/main/downloads

En onlineversion finns tillgänglig på:

https://gibbok.github.io/typescript-book

Innehållsförteckning

- The Concise TypeScript Book
  - Stöd projektet
  - Översättningar
  - Nedladdningar och webbplats
  - Innehållsförteckning
  - Introduktion
  - Om författaren
  - Introduktion till TypeScript
    - Vad är TypeScript?
    - Varför TypeScript?
    - TypeScript och JavaScript
    - TypeScript-kodgenerering
    - Modernt JavaScript nu (Downleveling)
  - Komma igång med TypeScript
    - Installation
    - Konfiguration
    - TypeScript-konfigurationsfil
      - target
      - lib
      - strict
      - module
      - moduleResolution

        - esModuleInterop
        - jsx
        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - Råd vid migrering till TypeScript
- Utforska typsystemet
    - TypeScript-språktjänsten
    - Strukturell typning
    - Grundläggande jämförelseregler i TypeScript
    - Typer som mängder
    - Tilldela en typ: Typdeklarationer och Typpåståenden
        - Typdeklaration
        - Typpåstående
        - Omgivande deklarationer
    - Egenskapskontroll och kontroll av överskottsegenskaper
    - Svaga typer
    - Strikt kontroll av objektliteraler (Freshness)
    - Typinferens
    - Mer avancerade inferenser
    - Typbreddning
    - Const
        - Const-modifierare på typparametrar
        - Const-påstående
    - Explicit typannotering
    - Typavsmalnande
        - Villkor
        - Kasta eller returnera

        - Diskriminerad union
        - Användardefinierade typvakter
        - Switch-true-förträngning
    - Primitiva typer
        - string
        - boolean
        - number
        - bigInt
        - Symbol
        - null och undefined
        - Array
        - any
    - Typannoteringar
    - Valfria egenskaper
    - Skrivskyddade egenskaper
    - Indexsignaturer
    - Utöka typer
    - Literaltyper
    - Literalhärledning
    - strictNullChecks
    - Enums
        - Numeriska enums
        - Sträng-enums
        - Konstanta enums
        - Omvänd mappning
        - Omgivande enums
        - Beräknade och konstanta medlemmar
    - Avsmalning
        - typeof-typvakter
        - Sanningsvärdesavsmalning
        - Likhetsavsmalning
        - In-operatoravsmalning

        - instanceof-avsmalning
    - Tilldelningar
    - Kontrollflödesanalys
    - Typpredikat
    - Diskriminerade unioner
    - Never-typen
    - Uttömmande kontroll
    - Objekttyper
    - Tuppeltyp (Anonym)
    - Namngiven tuppeltyp (Märkt)
    - Tuppel med fast längd
    - Unionstyp
    - Intersektionstyper
    - Typindexering
    - Typ från värde
    - Typ från funktionsreturvärde
    - Typ från modul
    - Mappade typer
    - Modifierare för mappade typer
    - Villkorliga typer
    - Distributiva villkorliga typer
    - infer-typinferens i villkorliga typer
    - Fördefinierade villkorliga typer
    - Mall-unionstyper
    - Any-typen
    - Unknown-typen
    - Void-typen
    - Användning av Never-typen
    - Interface och Type
        - Gemensam syntax
        - Grundläggande typer
        - Objekt och Interface

        - Union- och Intersection-typer
    - Inbyggda primitiva typer
    - Vanliga inbyggda JS-objekt
    - Överlagringar
    - Sammanslagning och utökning
    - Skillnader mellan Type och Interface
    - Klass
        - Vanlig klasssyntax
        - Konstruktor
        - Privata och skyddade konstruktorer
        - Åtkomstmodifierare
        - Get och Set
        - Auto-accessorer i klasser
        - this
        - Parameteregenskaper
        - Abstrakta klasser
        - Med generics
        - Dekoratörer
            - Klassdekoratörer
            - Egenskapsdekoratör
            - Metoddekoratör
            - Getter- och setter-dekoratörer
            - Dekoratörmetadata
        - Arv
        - Statiska medlemmar
        - Egenskapsinitiering
        - Metodöverlagring
    - Generics
        - Generisk typ
        - Generiska klasser
        - Generiska begränsningar
        - Generisk kontextuell avsmalning

- Raderade strukturella typer
- Namnrymder
- Symboler
- Trippelsnedstreck-direktiv
- Typmanipulation
  - Skapa typer från typer
  - Indexerade åtkomsttyper
  - Verktygstyper
    - Awaited<T>
    - Partial<T>
    - Required<T>
    - Readonly<T>
    - Record<K, T>
    - Pick<T, K>
    - Omit<T, K>
    - Exclude<T, U>
    - Extract<T, U>
    - NonNullable<T>
    - Parameters<T>
    - ConstructorParameters<T>
    - ReturnType<T>
    - InstanceType<T>
    - ThisParameterType<T>
    - OmitThisParameter<T>
    - ThisType<T>
    - Uppercase<T>
    - Lowercase<T>
    - Capitalize<T>
    - Uncapitalize<T>
    - NoInfer<T>
- Övrigt
  - Fel och undantagshantering

- Mixin-klasser
- Asynkrona språkfunktioner
- Iteratorer och generatorer
- TsDocs JSDoc-referens
- @types
- JSX
- ES6-moduler
- ES7 exponentiationsoperator
- for-await-of-satsen
- Metaegenskapen new target
- Dynamiska importuttryck
- “tsc –watch”
- Non-null Assertion Operator
- Standarddeklarationer
- Valfri kedjning (Optional Chaining)
- Nullish coalescing-operatorn
- Mallsträngslitteraltyper (Template Literal Types)
- Funktionsöverlagring
- Rekursiva typer
- Rekursiva villkorstyper
- Stöd för ECMAScript-moduler i Node
- Assertionsfunktioner
- Variadiska tuppeltyper
- Inkapslingstyper (Boxed types)
- Kovarians och kontravarians i TypeScript
  - Valfria variansannotationer för typparametrar
- Mallsträngsmönsterindexsignaturer
- satisfies-operatorn
- Importer och exporter av enbart typer
- using-deklaration och explicit resurshantering
  - await using-deklaration
- Importattribut

- Syntaxkontroll för reguljära uttryck
- import defer

Introduktion

Välkommen till The Concise TypeScript Book! Denna guide ger dig viktig kunskap och praktiska färdigheter för effektiv TypeScript-utveckling. Upptäck nyckelkoncept och tekniker för att skriva ren, robust kod. Oavsett om du är nybörjare eller en erfaren utvecklare fungerar denna bok både som en omfattande guide och en praktisk referens för att utnyttja TypeScripts kraft i dina projekt.

Denna bok täcker TypeScript 7.0.

Om författaren

Simone Poggiali är en erfaren Staff Engineer med en passion för att skriva professionell kod sedan 90-talet. Under sin internationella karriär har han bidragit till många projekt för ett brett spektrum av kunder, från startups till stora organisationer. Framstående företag som HelloFresh, Siemens, O2, Leroy Merlin och Snowplow har dragit nytta av hans expertis och engagemang.

Du kan nå Simone Poggiali på följande plattformar:

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- E-post: gibbok.coding gmail.com

Fullständig lista över bidragsgivare: https://github.com/gibbok/typescript-book/graphs/contributors

Introduktion till TypeScript

Vad är TypeScript?

TypeScript är ett starkt typat programmeringsspråk som bygger på JavaScript. Det designades ursprungligen av Anders Hejlsberg 2012 och utvecklas och underhålls för närvarande av Microsoft som ett projekt med öppen källkod.

TypeScript kompileras till JavaScript och kan köras i vilken JavaScript-runtime som helst (t.ex. en webbläsare eller Node.js på en server).

Det stöder flera programmeringsparadigm såsom funktionell, generisk, imperativ och objektorienterad programmering, och är ett kompilerat (transpilerat) språk som konverteras till JavaScript före exekvering.

Varför TypeScript?

TypeScript är ett starkt typat språk som hjälper till att förhindra vanliga programmeringsmisstag och undvika vissa typer av körtidsfel innan programmet körs.

Ett starkt typat språk gör det möjligt för utvecklaren att specificera olika programbegränsningar och beteenden i datatypsdefinitionerna, vilket underlättar möjligheten att verifiera programvarans korrekthet och förhindra defekter. Detta är särskilt värdefullt i storskaliga applikationer.

Några av fördelarna med TypeScript:

- Statisk typning, valfritt starkt typad
- Typinferens
- Tillgång till ES6- och ES7-funktioner
- Plattforms- och webbläsarkompatibilitet
- Verktygsstöd med IntelliSense

TypeScript och JavaScript

TypeScript skrivs i `.ts`- eller `.tsx`-filer, medan JavaScript-filer skrivs i `.js`- eller `.jsx`-filer.

Filer med tillägget `.tsx` eller `.jsx` kan innehålla JavaScript Syntax Extension JSX, som används i React för UI-utveckling.

TypeScript är en typad supermängd av JavaScript (ECMAScript 2015) vad gäller syntax. All JavaScript-kod är giltig TypeScript-kod, men det omvända är inte alltid sant.

Betrakta till exempel en funktion i en JavaScript-fil med tillägget `.js`, som följande:

```
const sum = (a, b) => a + b;
```

Funktionen kan konverteras och användas i TypeScript genom att ändra filtillägget till `.ts`. Men om samma funktion annoteras med TypeScript-typer kan den inte köras i någon JavaScript-runtime utan kompilering. Följande TypeScript-kod kommer att producera ett syntaxfel om den inte kompileras:

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript designades för att upptäcka möjliga undantag som kan uppstå vid körning under kompileringstiden genom att låta utvecklaren definiera avsikten med typannoteringar. Dessutom kan TypeScript också fånga problem om ingen typannotering tillhandahålls. Till exempel specificerar följande kodavsnitt inga TypeScript-typer:

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

I detta fall upptäcker TypeScript ett fel och rapporterar:

```
Property 'y' does not exist on type '{ x: number; }'.
```

TypeScripts typsystem är till stor del influerat av körtidsbeteendet hos JavaScript. Till exempel kan additionsoperatorn (+), som i JavaScript kan utföra antingen strängkonkatenering eller numerisk addition, modelleras på samma sätt i TypeScript:

```
const result = '1' + 1; // Resultatet har typen string
```

Teamet bakom TypeScript har fattat ett medvetet beslut att flagga ovanlig användning av JavaScript som fel. Betrakta till exempel följande giltiga JavaScript-kod:

```
const result = 1 + true; // I JavaScript är resultatet lika med
2
```

Dock kastar TypeScript ett fel:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Detta fel uppstår eftersom TypeScript strikt upprätthåller typkompatibilitet, och i detta fall identifierar det en ogiltig operation mellan en number och en boolean.

TypeScript-kodgenerering

TypeScript-kompilatorn har två huvudansvar: kontrollera typfel och kompilera till JavaScript. Dessa två processer är oberoende av varandra. Typer påverkar inte kodens exekvering i en JavaScript-runtime, eftersom de raderas helt under kompileringen. TypeScript kan fortfarande producera JavaScript även vid typfel. Här är ett exempel på TypeScript-kod med ett typfel:

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Ett argument av typen 'string' kan inte tilldelas en parameter av typen 'number'.
```

Trots detta kan den fortfarande producera körbar JavaScript-utdata:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

Det är inte möjligt att kontrollera TypeScript-typer vid körning. Till exempel:

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // 'Dog' hänvisar endast till en typ, men används här som ett värde.
```

```
        // ...
    }
};
```

Eftersom typerna raderas efter kompilering finns det inget sätt att köra denna kod i JavaScript. För att känna igen typer vid körning behöver vi använda en annan mekanism. TypeScript erbjuder flera alternativ, där ett vanligt är “tagged union”. Till exempel:

```
interface Dog {
    kind: 'dog'; // Diskriminerad union
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // Diskriminerad union
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

Egenskapen “kind” är ett värde som kan användas vid körning för att särskilja mellan objekt i JavaScript.

Det är också möjligt att ett värde vid körning har en annan typ än den som deklarerades i typdeklarationen. Till exempel om utvecklaren har misstolkat en API-typ och annoterat den felaktigt.

TypeScript är en supermängd av JavaScript, så nyckelordet "class" kan användas som en typ och ett värde vid körning.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};
```

```
const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

I JavaScript har en “class” en “prototype”-egenskap, och “instanceof”-operatorn kan användas för att testa om prototype-egenskapen för en konstruktor förekommer någonstans i prototypkedjan för ett objekt.

TypeScript har ingen effekt på körtidsprestanda, eftersom alla typer raderas. Dock introducerar TypeScript viss overhead vid byggtid.

Modernt JavaScript nu (Downleveling)

TypeScript kan kompilera kod till vilken utgiven version av JavaScript som helst sedan ECMAScript 3 (1999). Detta innebär att TypeScript kan transpilera kod från de senaste JavaScript-funktionerna till äldre versioner, en process som kallas Downleveling. Detta möjliggör användning av modernt JavaScript samtidigt som maximal kompatibilitet med äldre körtidsmiljöer bibehålls.

Det är viktigt att notera att vid transpilering till en äldre version av JavaScript kan TypeScript generera kod som kan medföra en prestandaoverhead jämfört med nativa implementeringar.

Här är några av de moderna JavaScript-funktioner som kan användas i TypeScript:

- ECMAScript-moduler istället för AMD-style “define”-callbacks eller CommonJS “require”-satser.
- Klasser istället för prototyper.
- Variabeldeklaration med “let” eller “const” istället för “var”.

- “for-of”-loop eller “.forEach” istället för den traditionella “for”-loopen.
- Pilfunktioner istället för funktionsuttryck.
- Destruktureringstilldelning.
- Förkortade egenskaps-/metodnamn och beräknade egenskapsnamn.
- Standardparametrar för funktioner.

Genom att utnyttja dessa moderna JavaScript-funktioner kan utvecklare skriva mer uttrycksfull och koncis kod i TypeScript.

Komma igång med TypeScript

Installation

Visual Studio Code erbjuder utmärkt stöd för TypeScript-språket men inkluderar inte TypeScript-kompilatorn. För att installera TypeScript-kompilatorn kan du använda en pakethanterare som npm eller yarn:

```
npm install typescript --save-dev
```

eller

```
yarn add typescript --dev
```

Se till att committa den genererade lockfilen för att säkerställa att varje teammedlem använder samma version av TypeScript.

För att köra TypeScript-kompilatorn kan du använda följande kommandon

```
npx tsc
```

eller

```
yarn tsc
```

Det rekommenderas att installera TypeScript projektvis snarare än globalt, eftersom det ger en mer förutsägbar byggprocess. För enstaka tillfällen kan du dock använda följande kommando:

```
npx tsc
```

eller installera det globalt:

```
npm install -g typescript
```

Om du använder Microsoft Visual Studio kan du hämta TypeScript som ett paket i NuGet för dina MSBuild-projekt. I NuGet Package Manager Console kör du följande kommando:

```
Install-Package Microsoft.TypeScript.MSBuild
```

Under TypeScript-installationen installeras två körbara filer: “tsc” som TypeScript-kompilatorn och “tsserver” som den fristående TypeScript-servern. Den fristående servern innehåller kompilatorn och språktjänster som kan användas av redigerare och IDE:er för att tillhandahålla intelligent kodkomplettering.

Dessutom finns det flera TypeScript-kompatibla transpilerare tillgängliga, såsom Babel (via ett plugin) eller swc. Dessa transpilerare kan användas för att konvertera TypeScript-kod till andra målspråk eller versioner.

TypeScript 7.0 skrevs om i Go som en nativ implementation av kompilatorn och språktjänsten. Den använder flertrådning med delat minne och andra optimeringar för att göra fullständiga

byggen och redigerarfunktioner snabbare, vilket minskar återkopplingstiden under utveckling.

Vissa prestandafunktioner i TypeScript 7.0 kan justeras. Typkontroll kan köras i parallella arbetare med `--checkers`; fler arbetare kan snabba upp stora projekt men använder mer minne. Det ombyggda `--watch`-läget förbättrar plattformsoberoende filbevakning. TypeScript 7.0 innehåller ännu inget compiler-API (från och med juli 2026), så verktyg som fortfarande behöver TypeScript 6.0-API:t kan köras sida vid sida med TypeScript 7.0 genom att använda `@typescript/typescript6` eller npm-alias.

Konfiguration

TypeScript kan konfigureras med hjälp av tsc CLI-alternativ eller genom att använda en dedikerad konfigurationsfil kallad tsconfig.json som placeras i projektets rot.

För att generera en tsconfig.json-fil förfylld med rekommenderade inställningar kan du använda följande kommando:

```
tsc --init
```

När kommandot `tsc` körs lokalt kommer TypeScript att kompilera koden med den konfiguration som anges i den närmaste tsconfig.json-filen.

Här är några exempel på CLI-kommandon som körs med standardinställningarna:

```
tsc main.ts // Kompilera en specifik fil (main.ts) till JavaScript
```

```
tsc src/*.ts // Kompilera alla .ts-filer i mappen 'src' till JavaScript
tsc app.ts util.ts --outfile index.js // Kompilera två TypeScript-filer (app.ts och util.ts) till en enda JavaScript-fil (index.js)
```

TypeScript-konfigurationsfil

En tsconfig.json-fil används för att konfigurera TypeScript-kompilatorn (tsc). Vanligtvis läggs den till i projektets rot, tillsammans med filen `package.json`.

Observera:

- tsconfig.json accepterar kommentarer även om det är i json-format.
- Det är tillrådligt att använda denna konfigurationsfil istället för kommandoradsalternativ.

På följande länk hittar du den fullständiga dokumentationen och dess schema:

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

Följande representerar en lista över de vanligaste och mest användbara konfigurationerna:

target

Egenskapen “target” används för att ange vilken ECMAScript-version din TypeScript-kod ska generera/kompilera till. För

moderna webbläsare är ES6 ett bra alternativ. Obs: ES5-stöd fasades ut i TypeScript 6.0 och stöds inte längre i TypeScript 7.0.

lib

Egenskapen “lib” används för att ange vilka biblioteksfiler som ska inkluderas vid kompilering. TypeScript inkluderar automatiskt API:er för funktioner som anges i “target”-egenskapen, men det är möjligt att utelämna eller välja specifika bibliotek för särskilda behov. Till exempel, om du arbetar med ett serverprojekt kan du exkludera “DOM”-biblioteket, som bara är användbart i en webbläsarmiljö.

strict

Alternativet “strict” förbättrar typsäkerheten genom att möjliggöra starkare kontroller. Det är aktiverat som standard från och med TypeScript 6.0; annars bör du explicit ställa in det till true i din tsconfig.json. Genom att aktivera “strict” kan TypeScript:

- Genererar kod med “use strict” för varje källfil.
- Beaktar “null” och “undefined” i typkontrollprocessen.
- Inaktiverar användningen av typen “any” när inga typannoteringar finns.
- Ger ett fel vid användning av “this”-uttrycket, som annars skulle innebära typen “any”.

module

Egenskapen “module” anger vilket modulsystem som stöds för det kompilerade programmet. Under körning används en

modulladdare för att hitta och köra beroenden baserat på det angivna modulsystemet.

De vanligaste modulladdarna som används i JavaScript är Node.js CommonJS för serverapplikationer och RequireJS för AMD-moduler i webbläsarbaserade webbapplikationer. TypeScript kan generera kod för olika modulsystem, inklusive UMD, System, ESNext, ES2015/ES6 och ES2020. Modulsystemet bör väljas baserat på målmiljön och den modulladdningsmekanism som är tillgänglig i den miljön.

Obs: Stöd för äldre modulsystem (AMD, UMD, SystemJS) fasades ut i TypeScript 6.0 och stöds inte längre i TypeScript 7.0.

moduleResolution

Egenskapen “moduleResolution” anger strategin för modulupplösning. Använd “nodenext” eller “bundler” för modern TypeScript-kod. Strategin “classic” används bara för gamla versioner av TypeScript (före 1.6).

esModuleInterop

Egenskapen “esModuleInterop” gör det möjligt att importera standard från CommonJS-moduler som inte exporterade med “default”-egenskapen. Denna egenskap tillhandahåller en shim för att säkerställa kompatibilitet i den genererade JavaScript-koden. Efter att ha aktiverat detta alternativ kan vi använda `import MyLibrary from "my-library"` istället för `import * as MyLibrary from "my-library"`.

“esModuleInterop” var ursprungligen ett alternativ för att undvika att ändringar skulle gå fel, men har länge varit rekommenderade standardinställningar. Att inaktivera dem kan orsaka subtila problem under körning när CommonJS används med ESM. Obs: Från och med TypeScript 6.0 är detta säkrare interoperabilitetsbeteende alltid aktiverat.

I TypeScript 6.0 fasades vissa äldre konfigurationsalternativ och syntaxformer ut eller övergick via gammalt beteende. I TypeScript 7.0 är de hårda fel eller no-op-beteende.

De utfasningar som har blivit hårda fel med no-op-beteende är:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- att inaktivera `esModuleInterop` eller `allowSyntheticDefaultImports`
- att inaktivera `alwaysStrict`
- nyckelordet `module` i namespace-deklarationer
- `asserts` på importer
- `/// <reference no-default-lib />` under `skipDefaultLibCheck`
- CLI-filsökvägar med en lokal `tsconfig.json`, om inte `--ignoreConfig` används

jsx

Egenskapen “jsx” gäller bara för .tsx-filer som används i ReactJS och styr hur JSX-konstruktioner kompileras till JavaScript. Ett

vanligt alternativ är “preserve” som kompilerar till en .jsx-fil och behåller JSX oförändrat så att det kan skickas vidare till olika verktyg som Babel för ytterligare transformationer.

skipLibCheck

Egenskapen “skipLibCheck” hindrar TypeScript från att typkontrollera hela importerade tredjepartspaket. Denna egenskap minskar kompileringstiden för ett projekt. TypeScript kommer fortfarande att kontrollera din kod mot typdefinitionerna som tillhandahålls av dessa paket.

files

Egenskapen “files” anger för kompilatorn en lista med filer som alltid måste inkluderas i programmet.

include

Egenskapen “include” anger för kompilatorn en lista med filer som vi vill inkludera. Denna egenskap tillåter glob-liknande mönster, såsom “*” *för valfri underkatalog,* ”” för valfritt filnamn och “?” för valfria tecken.

exclude

Egenskapen “exclude” anger för kompilatorn en lista med filer som inte bör inkluderas i kompileringen. Detta kan inkludera filer som “node_modules” eller testfiler. Observera: tsconfig.json tillåter kommentarer.

importHelpers

TypeScript använder hjälpkod när det genererar kod för vissa avancerade eller nedåtkompatibla JavaScript-funktioner. Som standard dupliceras dessa hjälpfunktioner i filer som använder dem. Alternativet `importHelpers` importerar dessa hjälpfunktioner från modulen `tslib` istället, vilket gör JavaScript-utdata mer effektiv.

Råd vid migrering till TypeScript

För stora projekt rekommenderas en gradvis övergång där TypeScript- och JavaScript-kod initialt samexisterar. Bara små projekt kan migreras till TypeScript på en gång.

Det första steget i denna övergång är att introducera TypeScript i byggkedjeprocessen. Detta kan göras genom att använda kompilatoralternativet “allowJs”, som tillåter att .ts- och .tsx-filer samexisterar med befintliga JavaScript-filer. Eftersom TypeScript faller tillbaka till typen “any” för en variabel när det inte kan härleda typen från JavaScript-filer, rekommenderas det att inaktivera “noImplicitAny” i dina kompilatoralternativ i början av migreringen.

Det andra steget är att säkerställa att dina JavaScript-tester fungerar tillsammans med TypeScript-filer så att du kan köra tester allt eftersom du konverterar varje modul. Om du använder Jest, överväg att använda `ts-jest`, som gör det möjligt att testa TypeScript-projekt med Jest.

Det tredje steget är att inkludera typdeklarationer för tredjepartsbibliotek i ditt projekt. Dessa deklarationer kan hittas antingen medföljande eller på DefinitelyTyped. Du kan söka efter dem med https://www.typescriptlang.org/dt/search och installera dem med:

```
npm install --save-dev @types/package-name
```

eller

```
yarn add --dev @types/package-name
```

Det fjärde steget är att migrera modul för modul med en nedifrån-och-upp-metod, genom att följa ditt beroendeträd med start från löven. Tanken är att börja konvertera moduler som inte är beroende av andra moduler. För att visualisera beroendegrafer kan du använda verktyget “madge”.

Bra kandidatmoduler för dessa initiala konverteringar är hjälpfunktioner och kod relaterad till externa API:er eller specifikationer. Det är möjligt att automatiskt generera TypeScript-typdefinitioner från Swagger-kontrakt, GraphQL- eller JSON-scheman som kan inkluderas i ditt projekt.

När det inte finns några specifikationer eller officiella scheman tillgängliga kan du generera typer från rådata, såsom JSON som returneras av en server. Det rekommenderas dock att generera typer från specifikationer istället för data för att undvika att missa specialfall.

Under migreringen bör du avstå från kodrefaktorisering och fokusera enbart på att lägga till typer i dina moduler.

Det femte steget är att aktivera “noImplicitAny”, vilket kommer att se till att alla typer är kända och definierade, vilket ger en bättre TypeScript-upplevelse för ditt projekt.

Under migreringen kan du använda direktivet `@ts-check`, som aktiverar TypeScript-typkontroll i en JavaScript-fil. Detta direktiv tillhandahåller en lös version av typkontroll och kan

initialt användas för att identifiera problem i JavaScript-filer. När `@ts-check` inkluderas i en fil kommer TypeScript att försöka härleda definitioner med hjälp av kommentarer i JSDoc-stil. Överväg dock att använda JSDoc-annoteringar bara i ett mycket tidigt skede av migreringen.

Överväg att behålla standardvärdet för `noEmitOnError` i din tsconfig.json som false. Detta gör att du kan generera JavaScript-källkod även om fel rapporteras.

Utforska typsystemet

TypeScript-språktjänsten

TypeScript-språktjänsten, även känd som tsserver, erbjuder olika funktioner såsom felrapportering, diagnostik, kompilera-vid-sparning, namnbyte, gå till definition, kompletteringslistor, signaturhjälp och mer. Den används främst av integrerade utvecklingsmiljöer (IDE:er) för att ge IntelliSense-stöd. Den integreras sömlöst med Visual Studio Code och används av verktyg som Conquer of Completion (Coc).

Utvecklare kan utnyttja ett dedikerat API och skapa sina egna anpassade språktjänstplugin för att förbättra TypeScript-redigeringsupplevelsen. Detta kan vara särskilt användbart för att implementera speciella linting-funktioner eller möjliggöra automatisk komplettering för ett anpassat mallspråk.

Ett exempel på ett verkligt anpassat plugin är “typescript-styled-plugin”, som tillhandahåller syntaxfelrapportering och IntelliSense-stöd för CSS-egenskaper i styled components.

För mer information och snabbstartsguider kan du hänvisa till den officiella TypeScript-wikin på GitHub: https://github.com/microsoft/TypeScript/wiki/

Strukturell typning

TypeScript är baserat på ett strukturellt typsystem. Detta innebär att kompatibiliteten och ekvivalensen hos typer bestäms av typens faktiska struktur eller definition, snarare än dess namn eller plats för deklaration, som i nominativa typsystem som C# eller C.

TypeScripts strukturella typsystem designades baserat på hur JavaScripts dynamiska duck typing-system fungerar vid körning.

Följande exempel är giltig TypeScript-kod. Som du kan observera har "X" och "Y" samma medlem "a", även om de har olika deklarationsnamn. Typerna bestäms av deras strukturer, och i detta fall, eftersom strukturerna är desamma, är de kompatibla och giltiga.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // Giltigt
```

Grundläggande jämförelseregler i TypeScript

TypeScripts jämförelseprocess är rekursiv och utförs på typer som är nästlade på valfri nivå.

En typ "X" är kompatibel med "Y" om "Y" har åtminstone samma medlemmar som "X".

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // Giltigt, eftersom det har åtminstone samma medlemmar som X
const r: X = y;
```

Funktionsparametrar jämförs efter typer, inte efter deras namn:

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Giltigt
x = y; // Giltigt
```

Funktionens returtyper måste vara desamma:

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Ogiltigt
x = y; // Ogiltigt
```

Returtypen för en källfunktion måste vara en undertyp av returtypen för en målfunktion:

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Giltigt
y = x; // Ogiltigt, medlemmen b saknas
```

Att utelämna funktionsparametrar är tillåtet, eftersom det är vanlig praxis i JavaScript, till exempel vid användning av “Array.prototype.map()”:

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Därför är följande typdeklarationer helt giltiga:

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Parametern b saknas
y = x; // Giltigt
```

Eventuella ytterligare valfria parametrar i källtypen är giltiga:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Giltigt
x = y; //Giltigt
```

Eventuella valfria parametrar i måltypen utan motsvarande parametrar i källtypen är giltiga och utgör inte ett fel:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Giltigt
x = y; // Giltigt
```

Rest-parametern behandlas som en oändlig serie av valfria parametrar:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //giltigt
```

Funktioner med överlagringar är giltiga om överlagringssignaturen är kompatibel med dess implementeringssignatur:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Giltigt
x('a', 1); // Giltigt

function y(a: string): void; // Ogiltigt, inte kompatibelt med
implementationssignaturen
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
y('a');
y('a', 1);
```

Jämförelse av funktionsparametrar lyckas om käll- och målparametrarna kan tilldelas supertyper eller undertyper (bivarians).

```
// Supertyp
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Undertyp
class Y extends X {}
// Undertyp
class Z extends X {}

type GetA = (x: X) => string;
```

```
const getA: GetA = x => x.a;

// Bivarians accepterar supertyper
console.log(getA(new X('x'))); // Giltigt
console.log(getA(new Y('Y'))); // Giltigt
console.log(getA(new Z('z'))); // Giltigt
```

Enums är jämförbara och giltiga med tal och vice versa, men att jämföra Enum-värden från olika Enum-typer är ogiltigt.

```
enum X {
    A,
    B,
}
enum Y {
    A,
    B,
    C,
}
const xa: number = X.A; // Giltigt
const ya: Y = 0; // Giltigt
X.A === Y.A; // Ogiltigt
```

Instanser av en klass genomgår en kompatibilitetskontroll för sina privata och skyddade medlemmar:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
```

```
}

let x: X = new Y('y'); // Ogiltigt
```

Jämförelsekontrollen tar inte hänsyn till den olika arvshierarkin, till exempel:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Giltigt
x === z; // Giltigt även om z tillhör en annan arvshierarki
```

Generics jämförs med hjälp av deras strukturer baserat på den resulterande typen efter tillämpning av den generiska parametern. Bara slutresultatet jämförs som en icke-generisk typ.

```ts
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Ogiltigt eftersom typargumentet används i den slutliga strukturen

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Giltigt eftersom typargumentet inte används i den slutliga strukturen
```

När generics inte har sitt typargument specificerat behandlas alla ospecificerade argument som typer med “any”:

```ts
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Giltigt
```

Kom ihåg:

```ts
let a: number = 1;
let b: number = 2;
a = b; // Giltigt, allt kan tilldelas sig självt

let c: any;
c = 1; // Giltigt, alla typer kan tilldelas any

let d: unknown;
d = 1; // Giltigt, alla typer kan tilldelas unknown

let e: unknown;
let e1: unknown = e; // Giltigt, unknown kan endast tilldelas sig självt och any
let e2: any = e; // Giltigt
```

```
let e3: number = e; // Ogiltigt

let f: never;
f = 1; // Ogiltigt, inget kan tilldelas never

let g: void;
let g1: any;
g = 1; // Ogiltigt, void kan varken tilldelas eller ta emot något annat än any
g = g1; // Giltigt
```

Observera att när "strictNullChecks" är aktiverat behandlas "null" och "undefined" på liknande sätt som "void"; annars liknar de "never".

Typer som mängder

I TypeScript är en typ en mängd av möjliga värden. Denna mängd kallas även typens domän. Varje värde av en typ kan ses som ett element i en mängd. En typ fastställer de begränsningar som varje element i mängden måste uppfylla för att betraktas som en medlem av den mängden. TypeScripts primära uppgift är att kontrollera och verifiera om en mängd är en delmängd av en annan.

TypeScript stöder olika typer av mängder:

Mängdterm	TypeScript	Anteckningar
Tom mängd	never	"never" innehåller ingenting förutom sig själv
Enelement-mängd	undefined / null / literal type	
Ändlig mängd	boolean / union	

Oändlig mängd	string / number / object	
Universell mängd	any / unknown	Varje element är medlem i “any” och varje mängd är en delmängd av den / “unknown” är en typsäker motsvarighet till “any”

Här är några exempel:

TypeScript	Mängdterm	Exempel
never	∅ (tom mängd)	const x: never = ‘x’; // Error: Type ‘string’ is not assignable to type ‘never’
Literal type	Enelement-mängd	type X = ‘X’;
		type Y = 7;
Värde tilldelbart till T	Värde ∈ T (medlem av)	type XY = ‘X’ | ‘Y’;
		const x: XY = ‘X’;
T1 tilldelbart till T2	T1 ⊆ T2 (delmängd av)	type XY = ‘X’ | ‘Y’;
		const x: XY = ‘X’;
		const j: XY = ‘J’; // Type ‘“J”’ is not assignable to type ‘XY’.
T1 extends T2	T1 ⊆ T2 (delmängd av)	type X = ‘X’ extends string ? true : false;
T1 | T2	T1 ∪ T2 (union)	type XY = ‘X’ | ‘Y’;
		type JK = 1 | 2;
T1 & T2	T1 ∩ T2 (snitt)	type X = { a: string }
		type Y = { b: string }
		type XY = X & Y

		const x: XY = { a: ‘a’, b: ‘b’ }
unknown	Universell mängd	const x: unknown = 1

En union, (T1 | T2) skapar en bredare mängd (båda):

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Giltigt
```

En intersektion, (T1 & T2) skapar en smalare mängd (endast delade):

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Ogiltigt
const j: XY = { a: 'a', b: 'b' }; // Giltigt
```

Nyckelordet `extends` kan betraktas som “delmängd av” i detta sammanhang. Det anger en begränsning för en typ. När `extends` används med en generisk typ begränsar det den generiska typparametern till en mer specifik typ.

Observera att `extends` här inte har något att göra med klassarv i OOP-sammanhang.

TypeScript arbetar med strukturella typer och har inte en strikt nominell hierarki. Faktum är att, som i exemplet nedan, två typer kan överlappa utan att någon av dem är en subtyp av den andra, eftersom TypeScript tar hänsyn till strukturen, eller formen, hos objekt.

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Giltigt
```

Tilldela en typ: Typdeklarationer och Typpåståenden

En typ kan tilldelas på olika sätt i TypeScript:

Typdeklaration

I följande exempel använder vi x: X (“: Type”) för att deklarera en typ för variabeln x.

```
type X = {
    a: string;
};

// Typdeklaration
const x: X = {
    a: 'a',
};
```

Om variabeln inte har det angivna formatet kommer TypeScript att rapportera ett fel. Till exempel:

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
      b: 'b', // Fel: En objektliteral får endast ange kända egenskaper
};
```

Typpåstående

Det är möjligt att lägga till ett påstående genom att använda nyckelordet `as`. Detta talar om för kompilatorn att utvecklaren har mer information om en typ och tystar eventuella fel som kan uppstå.

Till exempel:

```
type X = {
    a: string;
```

```
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

I exemplet ovan påstås objektet x ha typen X med hjälp av nyckelordet as. Detta informerar TypeScript-kompilatorn om att objektet överensstämmer med den angivna typen, även om det har en extra egenskap b som inte finns i typdefinitionen.

Typpåståenden är användbara i situationer där en mer specifik typ behöver anges, särskilt vid arbete med DOM:en. Till exempel:

```
const myInput = document.getElementById('my_input') as HTMLInputElement;
```

Här används typpåståendet as HTMLInputElement för att tala om för TypeScript att resultatet av getElementById ska behandlas som ett HTMLInputElement. Typpåståenden kan också användas för att mappa om nycklar, som visas i exemplet nedan med malliteraler:

```
type J<Type> = {
    [
        Property in keyof Type as `prefix_${string & Property}`
    ]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

I detta exempel använder typen `J<Type>` en mappad typ med en malliteral för att mappa om nycklarna i Type. Den skapar nya egenskaper med ett "prefix_" tillagt till varje nyckel, och deras motsvarande värden är funktioner som returnerar de ursprungliga egenskapsvärdena.

Det är värt att notera att när man använder ett typpåstående kommer TypeScript inte att utföra kontroll av överskottsegenskaper. Därför är det generellt att föredra att använda en typdeklaration när objektets struktur är känd i förväg.

Omgivande deklarationer

Omgivande deklarationer är filer som beskriver typer för JavaScript-kod. De har filnamnsformatet `.d.ts`. De importeras vanligtvis och används för att annotera befintliga JavaScript-bibliotek eller för att lägga till typer till befintliga JS-filer i ditt projekt.

Många vanliga bibliotekstyper finns på: https://github.com/DefinitelyTyped/DefinitelyTyped/

och kan installeras med:

```
npm install --save-dev @types/library-name
```

För dina egna omgivande deklarationer kan du importera dem med "triple-slash"-referensen:

```
/// <reference path="./library-types.d.ts" />
```

Du kan använda omgivande deklarationer även i JavaScript-filer med `// @ts-check`.

Nyckelordet `declare` möjliggör typdefinitioner för befintlig JavaScript-kod utan att importera den, och fungerar som en platshållare för typer från en annan fil eller globalt.

Egenskapskontroll och kontroll av överskottsegenskaper

TypeScript bygger på ett strukturellt typsystem, men kontroll av överskottsegenskaper är en egenskap hos TypeScript som gör det möjligt att kontrollera om ett objekt har exakt de egenskaper som anges i typen.

Kontroll av överskottsegenskaper utförs vid tilldelning av objektliteraler till variabler eller när de skickas som argument till funktionens överskottsegenskap, till exempel.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
const x: X = y; // Giltigt på grund av strukturell typning
const w: X = { a: 'a', b: 'b' }; // Ogiltigt på grund av
kontrollen av överflödiga egenskaper
```

Svaga typer

En typ anses vara svag när den inte innehåller annat än en uppsättning helt valfria egenskaper:

```
type X = {
    a?: string;
    b?: string;
};
```

TypeScript betraktar det som ett fel att tilldela något till en svag typ när det inte finns någon överlappning. Till exempel ger följande ett fel:

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // Ogiltigt
```

Även om det inte rekommenderas, är det möjligt att kringgå denna kontroll genom att använda typpåstående om det behövs:

```
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Giltigt
```

Eller genom att lägga till `unknown` i indexsignaturen till den svaga typen:

```
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Giltigt
```

Strikt kontroll av objektliteraler (Freshness)

Strikt kontroll av objektliteraler, ibland kallad “freshness”, är en funktion i TypeScript som hjälper till att fånga överskotts- eller felstavade egenskaper som annars skulle gå obemärkta vid normala strukturella typkontroller.

När man skapar en objektliteral betraktar TypeScript-kompilatorn den som “fresh”. Om objektliteralen tilldelas till en variabel eller skickas som parameter kommer TypeScript att ge ett fel om objektliteralen anger egenskaper som inte finns i måltypen.

Dock försvinner “freshness” när en objektliteral breddas eller ett typpåstående används.

Här är några exempel för att illustrera:

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Färskhetskontroll: Ogiltig tilldelning
var y: Y;
y = { a: 'a', bx: 'bx' }; // Färskhetskontroll: Ogiltig tilldelning

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Typbreddning: Inga fel, strukturellt typkompatibel

fn({ a: 'a', bx: 'b' }); // Färskhetskontroll: Ogiltigt argument

let c: X = { a: 'a' };
```

```
let d: Y = { a: 'a', b: '' };
c = d; // Typbreddning: Ingen färskhetskontroll
```

Typinferens

TypeScript kan härleda typer när ingen annotering tillhandahålls vid:

- Variabelinitiering.
- Medlemsinitiering.
- Inställning av standardvärden för parametrar.
- Funktionens returtyp.

Till exempel:

```
let x = 'x'; // Den härledda typen är string
```

TypeScript-kompilatorn analyserar värdet eller uttrycket och bestämmer dess typ baserat på tillgänglig information.

Mer avancerade inferenser

När flera uttryck används vid typinferens letar TypeScript efter de “bästa gemensamma typerna”. Till exempel:

```
let x = [1, 'x', 1, null]; // Den härledda typen är: (string | number | null)[]
```

Om kompilatorn inte kan hitta de bästa gemensamma typerna returnerar den en unionstyp. Till exempel:

```
let x = [new RegExp('x'), new Date()]; // Den härledda typen är: (RegExp | Date)[]
```

TypeScript använder “kontextuell typning” baserat på variabelns placering för att härleda typer. I följande exempel

vet kompilatorn att `e` är av typen `MouseEvent` på grund av händelsetypen `click` som definieras i filen lib.d.ts, vilken innehåller omgivande deklarationer för olika vanliga JavaScript-konstruktioner och DOM:en:

```
window.addEventListener('click', function (e) {}); // Den härledda typen för e är MouseEvent
```

Typbreddning

Typbreddning är den process där TypeScript tilldelar en typ till en variabel som initierats utan att en typannotering angavs. Den tillåter övergång från smal till bredare typ men inte tvärtom. I följande exempel:

```
let x = 'x'; // TypeScript härleder string, en bred typ
let y: 'y' | 'x' = 'y'; // Typen för y är en union av literaltyper
y = x; // Ogiltigt: Typen 'string' kan inte tilldelas typen '"x" | "y"'.
```

TypeScript tilldelar `string` till `x` baserat på det enda värde som angavs vid initieringen (`x`), detta är ett exempel på breddning.

TypeScript tillhandahåller sätt att kontrollera breddningsprocessen, till exempel genom att använda “const”.

Const

Att använda nyckelordet `const` vid deklaration av en variabel resulterar i en smalare typinferens i TypeScript.

Till exempel:

```
const x = 'x'; // TypeScript härleder typen för x som 'x', en snävare typ
let y: 'y' | 'x' = 'y';
y = x; // Giltigt: Typen för x härleds som 'x'
```

Genom att använda `const` för att deklarera variabeln x, smalnas dess typ av till det specifika literalvärdet ‘x’. Eftersom typen av x är avsmalnad kan den tilldelas till variabeln y utan något fel. Anledningen till att typen kan härledas är att `const`-variabler inte kan omtilldelas, så deras typ kan smalnas av till en specifik literaltyp, i detta fall literaltypen ‘x’.

Const-modifierare på typparametrar

Från version 5.0 av TypeScript är det möjligt att ange attributet `const` på en generisk typparameter. Detta möjliggör härledning av den mest precisa typen möjligt. Låt oss se ett exempel utan att använda `const`:

```
function identity<T>(value: T) {
    // Inget const här
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Den härledda typen är: { a: string; b: string; }
```

Som du kan se härleddes egenskaperna `a` och `b` med typen `string`.

Låt oss nu se skillnaden med `const`-versionen:

```
function identity<const T>(value: T) {
    // Använder modifieraren const på typparametrar
    return value;
```

```
}
const values = identity({ a: 'a', b: 'b' }); // Den härledda
typen är: { a: "a"; b: "b"; }
```

Nu kan vi se att egenskaperna `a` och `b` härleddes som `const`, så `a` och `b` behandlas som strängliteraler snarare än bara `string`-typer.

Const-påstående

Denna funktion låter dig deklarera en variabel med en mer precis literaltyp baserat på dess initieringsvärde, och signalerar till kompilatorn att värdet ska behandlas som en oföränderlig literal. Här är några exempel:

På en enskild egenskap:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

På ett helt objekt:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

Detta kan vara särskilt användbart vid definition av typen för en tupel:

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Tupel readonly [1, 2, 3]
```

Explicit typannotering

Vi kan vara specifika och ange en typ. I följande exempel är egenskapen `x` av typen `number`:

```
const v = {
    x: 1, // Härledd typ: number (typbreddning)
};
v.x = 3; // Giltigt
```

Vi kan göra typannoteringen mer specifik genom att använda en union av literaltyper:

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x är nu en union av literaltyper: 1 | 2 | 3
};
v.x = 3; // Giltigt
v.x = 100; // Ogiltigt
```

Typavsmalnande

Typavsmalnande är den process i TypeScript där en generell typ smalnas av till en mer specifik typ. Detta sker när TypeScript analyserar koden och avgör att vissa villkor eller operationer kan förfina typinformationen.

Avsmalnande av typer kan ske på olika sätt, bland annat:

Villkor

Genom att använda villkorssatser, som `if` eller `switch`, kan TypeScript smalna av typen baserat på utfallet av villkoret. Till exempel:

```
let x: number | undefined = 10;

if (x !== undefined) {
```

```
        x += 100; // Typen är number, som har snävats in av
villkoret
}
```

Kasta eller returnera

Att kasta ett fel eller returnera tidigt från en gren kan användas för att hjälpa TypeScript smalna av en typ. Till exempel:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

Andra sätt att smalna av typer i TypeScript inkluderar:

- `instanceof`-operatorn: Används för att kontrollera om ett objekt är en instans av en specifik klass.
- `in`-operatorn: Används för att kontrollera om en egenskap finns i ett objekt.
- `typeof`-operatorn: Används för att kontrollera typen av ett värde vid körning.
- Inbyggda funktioner som `Array.isArray()`: Används för att kontrollera om ett värde är en array.

Diskriminerad union

Att använda en “diskriminerad union” är ett mönster i TypeScript där en explicit “tagg” läggs till objekt för att skilja mellan olika typer inom en union. Detta mönster kallas också en “taggad union”. I följande exempel representeras “taggen” av egenskapen “type”:

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // typen är A
        case 'type_b':
            return input.value + 'extra'; // typen är B
    }
};
```

Användardefinierade typvakter

I fall där TypeScript inte kan avgöra en typ är det möjligt att skriva en hjälpfunktion känd som en "användardefinierad typvakt". I följande exempel kommer vi att använda ett typpredikat för att smalna av typen efter att viss filtrering har tillämpats:

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // Typen är (string | null)[], TypeScript kunde inte härleda typen korrekt

const isValid = (item: string | null): item is string => item !== null; // Anpassad typvakt

const r2 = data.filter(isValid); // Typen är nu korrekt string[], med hjälp av typvakten med predikat kunde vi snäva in typen
```

Switch-true-förträngning

TypeScript 5.3 lägger till switch-true-förträngning, vilket låter dig ersätta röriga if/else-kedjor med switch (true) med hjälp av

booleska villkor. Det förbättrar läsbarheten och förtränger fortfarande typer. Det liknar mönstermatchning, men enklare.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

Primitiva typer

TypeScript stöder 7 primitiva typer. En primitiv datatyp avser en typ som inte är ett objekt och som inte har några metoder kopplade till sig. I TypeScript är alla primitiva typer oföränderliga, vilket innebär att deras värden inte kan ändras efter att de har tilldelats.

string

Den primitiva typen `string` lagrar textdata, och värdet omges alltid av dubbla eller enkla citattecken.

```
const x: string = 'x';
const y: string = 'y';
```

Strängar kan sträcka sig över flera rader om de omges av backtick-tecknet (`):

```
let sentence: string = `xxx,
   yyy`;
```

boolean

Datatypen `boolean` i TypeScript lagrar ett binärt värde, antingen `true` eller `false`.

```
const isReady: boolean = true;
```

number

En `number`-datatyp i TypeScript representeras med ett 64-bitars flyttalsvärde. En `number`-typ kan representera heltal och bråktal. TypeScript stöder även hexadecimala, binära och oktala tal, till exempel:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Hexadecimala tal börjar
med 0x
const binary: number = 0b1010; // Binära tal börjar med 0b
const octal: number = 0o633; // Oktala tal börjar med 0o
```

bigInt

En `bigInt` representerar numeriska värden som är mycket stora (253 – 1) och inte kan representeras med en `number`.

En `bigInt` kan skapas genom att anropa den inbyggda funktionen `BigInt()` eller genom att lägga till `n` i slutet av ett heltalsliteral:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Noteringar:

- `bigInt`-värden kan inte blandas med `number` och kan inte användas med inbyggd `Math`, de måste konverteras till samma typ.
- `bigInt`-värden är bara tillgängliga om målkonfigurationen är ES2020 eller högre.

Symbol

Symboler är unika identifierare som kan användas som egenskapsnycklar i objekt för att förhindra namnkonflikter.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null och undefined

`null` och `undefined`-typerna representerar båda inget värde eller frånvaron av något värde.

Typen `undefined` betyder att värdet inte är tilldelat eller initierat och indikerar en oavsiktlig frånvaro av värde.

Typen `null` betyder att vi vet att fältet inte har något värde, alltså är värdet otillgängligt, och det indikerar en avsiktlig frånvaro av värde.

Array

En `array` är en datatyp som kan lagra flera värden av samma typ eller inte. Den kan definieras med följande syntax:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Union
```

TypeScript stöder skrivskyddade arrayer med följande syntax:

```
const x: readonly string[] = ['a', 'b']; // Modifieraren readonly
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Ogiltigt
```

TypeScript stöder tupler och skrivskyddade tupler:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

Datatypen `any` representerar bokstavligen "vilket som helst" värde, det är standardvärdet när TypeScript inte kan härleda typen eller den inte är specificerad.

När `any` används hoppar TypeScript-kompilatorn över typkontrollen, så det finns ingen typsäkerhet när `any` används. Använd i allmänhet inte `any` för att tysta kompilatorn när ett fel uppstår, fokusera istället på att åtgärda felet eftersom det med `any` är möjligt att bryta kontrakt och vi förlorar fördelarna med TypeScripts autokomplettering.

Typen `any` kan vara användbar vid en gradvis migrering från JavaScript till TypeScript, eftersom den kan tysta kompilatorn.

För nya projekt, använd TypeScript-konfigurationen `noImplicitAny` som gör att TypeScript ger fel där `any` används eller härleds.

Typen `any` är vanligtvis en källa till fel som kan dölja verkliga problem med dina typer. Undvik att använda den så mycket som möjligt.

Typannoteringar

På variabler som deklareras med `var`, `let` och `const` är det möjligt att valfritt lägga till en typ:

```
const x: number = 1;
```

TypeScript är bra på att härleda typer, särskilt enkla sådana, så dessa deklarationer är i de flesta fall inte nödvändiga.

På funktioner är det möjligt att lägga till typannoteringar på parametrar:

```
function sum(a: number, b: number) {
    return a + b;
}
```

Följande är ett exempel med en anonym funktion (så kallad lambda-funktion):

```
const sum = (a: number, b: number) => a + b;
```

Dessa annoteringar kan undvikas när ett standardvärde för en parameter finns:

```
const sum = (a = 10, b: number) => a + b;
```

Returtypannoteringar kan läggas till på funktioner:

```
const sum = (a = 10, b: number): number => a + b;
```

Detta är särskilt användbart för mer komplexa funktioner eftersom det att explicit skriva returtypen före en implementation kan hjälpa till att bättre tänka igenom funktionen.

Generellt sett, överväg att annotera typsignaturer men inte lokala variabler i funktionskroppen, och lägg alltid till typer på objektliteraler.

Valfria egenskaper

Ett objekt kan specificera valfria egenskaper genom att lägga till ett frågetecken `?` i slutet av egenskapsnamnet:

```
type X = {
    a: number;
    b?: number; // Valfri
};
```

Det är möjligt att ange ett standardvärde när en egenskap är valfri:

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

Skrivskyddade egenskaper

Det är möjligt att förhindra skrivning till en egenskap genom att använda modifieraren `readonly` som ser till att egenskapen inte kan skrivas om, men den ger ingen garanti för total oföränderlighet:

```
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

Indexsignaturer

I TypeScript kan vi använda `string`, `number` och `symbol` som indexsignatur:

```
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // Samma resultat som k[1]
```

Observera att JavaScript automatiskt konverterar ett index med `number` till ett index med `string`, så `k[1]` eller `k["1"]` returnerar

samma värde.

Utöka typer

Det är möjligt att utöka ett `interface` (kopiera medlemmar från en annan typ):

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

Det är också möjligt att utöka från flera typer:

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

Nyckelordet `extends` fungerar bara på interface och klasser, för typer använd en intersektion:

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

Det är möjligt att utöka en `typ` med hjälp av ett `gränssnitt` men inte tvärtom:

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

Literaltyper

En literaltyp är en enskild elementuppsättning från en kollektiv typ, den definierar ett mycket exakt värde som är en JavaScript-primitiv.

Literaltyper i TypeScript är tal, strängar och booleaner.

Exempel på literaler:

```
const a = 'a'; // Strängliteraltyp
const b = 1; // Numerisk literaltyp
const c = true; // Boolesk literaltyp
```

Sträng-, numeriska och booleska literaltyper används i unioner, typvakter och typalias. I följande exempel kan du se ett unionstypealias. `O` består bara av de angivna värdena, ingen annan sträng är giltig:

```
type O = 'a' | 'b' | 'c';
```

Literalhärledning

Literalhärledning är en funktion i TypeScript som gör att typen av en variabel eller parameter kan härledas baserat på dess

värde.

I följande exempel kan vi se att TypeScript betraktar `x` som en literaltyp eftersom värdet inte kan ändras senare, medan `y` härleds som sträng eftersom det kan ändras när som helst.

```
const x = 'x'; // Literaltypen 'x', eftersom värdet inte kan ändras
let y = 'y'; // Typen string, eftersom värdet kan ändras
```

I följande exempel kan vi se att `o.x` härleds som en `string` (och inte en literal av `a`) eftersom TypeScript anser att värdet kan ändras när som helst.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // Detta är en bredare strängtyp
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // Ett argument av typen 'string' kan inte tilldelas en parameter av typen 'X'
```

Som du kan se ger koden ett fel när `o.x` skickas till `fn` eftersom X är en smalare typ.

Vi kan lösa detta problem genom att använda typbekräftelse med `const` eller typen `X`:

```
let o = {
    x: 'a' as const,
};
```

eller:

```
let o = {
    x: 'a' as X,
};
```

strictNullChecks

`strictNullChecks` är ett TypeScript-kompilatoralternativ som framtvingar strikt null-kontroll. När detta alternativ är aktiverat kan variabler och parametrar bara tilldelas `null` eller `undefined` om de uttryckligen har deklarerats att vara av den typen med hjälp av unionstypen `null | undefined`. Om en variabel eller parameter inte uttryckligen deklareras som nullable kommer TypeScript att generera ett fel för att förhindra potentiella körtidsfel.

Enums

I TypeScript är en `enum` en uppsättning namngivna konstantvärden.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

Enums kan definieras på olika sätt:

Numeriska enums

I TypeScript är en numerisk Enum en Enum där varje konstant tilldelas ett numeriskt värde, med start från 0 som standard.

```
enum Size {
    Small, // värdet börjar på 0
    Medium,
    Large,
}
```

Det är möjligt att ange anpassade värden genom att explicit tilldela dem:

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

Sträng-enums

I TypeScript är en sträng-Enum en Enum där varje konstant tilldelas ett strängvärde.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

Notera: TypeScript tillåter användning av heterogena Enums där sträng- och numeriska medlemmar kan samexistera.

Konstanta enums

En konstant enum i TypeScript är en speciell typ av Enum där alla värden är kända vid kompileringstid och infogas överallt där enum:en används, vilket resulterar i mer effektiv kod.

```
const enum Language {
    English = 'EN',
```

```
    Spanish = 'ES',
}
console.log(Language.English);
```

Kommer att kompileras till:

```
console.log('EN' /* Language.English */);
```

Noteringar: Konstanta Enums har hårdkodade värden som raderar Enum:en, vilket kan vara mer effektivt i fristående bibliotek men är i allmänhet inte önskvärt. Dessutom kan konstanta enums inte ha beräknade medlemmar.

Omvänd mappning

I TypeScript avser omvänd mappning i Enums möjligheten att hämta Enum-medlemmens namn från dess värde. Som standard har Enum-medlemmar framåtmappningar från namn till värde, men omvända mappningar kan skapas genom att explicit ange värden för varje medlem. Omvända mappningar är användbara när du behöver slå upp en Enum-medlem efter dess värde, eller när du behöver iterera över alla Enum-medlemmar. Observera att bara numeriska Enum-medlemmar genererar omvända mappningar, medan sträng-Enum-medlemmar inte genererar någon omvänd mappning alls.

Följande enum:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

Kompileras till:

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Därför fungerar mappning av värden till nycklar för numeriska enum-medlemmar, men inte för sträng-enum-medlemmar:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]); // Elementet har implicit typen
'any' eftersom indexuttrycket inte har typen 'number'.
```

Omgivande enums

En omgivande enum i TypeScript är en typ av Enum som definieras i en deklarationsfil (*.d.ts) utan en associerad implementation. Den låter dig definiera en uppsättning namngivna konstanter som kan användas på ett typsäkert sätt över olika filer utan att behöva importera implementationsdetaljerna i varje fil.

Beräknade och konstanta medlemmar

I TypeScript är en beräknad medlem en medlem av en Enum som har ett värde som beräknas vid körning, medan en konstant medlem är en medlem vars värde sätts vid kompileringstid och inte kan ändras under körning. Beräknade medlemmar är tillåtna i vanliga Enums, medan konstanta medlemmar är tillåtna i både vanliga och const enums.

```
// Konstanta medlemmar
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6, genererat vid kompilering

// Beräknade medlemmar
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // Slumptal genererat vid körning
```

Enums betecknas av unioner som består av deras medlemstyper. Värdena för varje medlem kan bestämmas genom konstanta eller icke-konstanta uttryck, där medlemmar med konstanta värden tilldelas literaltyper. För att illustrera, betrakta deklarationen av typ E och dess undertyper E.A, E.B och E.C. I detta fall representerar E unionen E.A | E.B | E.C.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Numerisk literal
```

```
    B = 'bar', // Strängliteral
    C = identity(42), // Opakt beräknat värde
}

console.log(E.C); //42
```

Avsmalning

TypeScript-avsmalning är processen att förfina typen av en variabel inom ett villkorsblock. Detta är användbart när man arbetar med unionstyper, där en variabel kan ha mer än en typ.

TypeScript känner igen flera sätt att avsmalna typen:

typeof-typvakter

typeof-typvakten är en specifik typvakt i TypeScript som kontrollerar typen av en variabel baserat på dess inbyggda JavaScript-typ.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x har typen number
    }
    return -1;
};
```

Sanningsvärdesavsmalning

Sanningsvärdesavsmalning i TypeScript fungerar genom att kontrollera om en variabel är sann eller falsk för att avsmalna dess typ därefter.

```
const toUpperCase = (name: string | null) => {
    if (name) {
```

```
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

Likhetsavsmalning

Likhetsavsmalning i TypeScript fungerar genom att kontrollera om en variabel är lika med ett specifikt värde eller inte, för att avsmalma dess typ därefter.

Den används tillsammans med `switch`-satser och likhetsoperatorer som `===`, `!==`, `==` och `!=` för att avsmalma typer.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

In-operatoravsmalning

`in`-operatoravsmalning i TypeScript är ett sätt att avsmalma typen av en variabel baserat på om en egenskap finns inom variabelns typ.

```
type Dog = {
    name: string;
    breed: string;
};

type Cat = {
    name: string;
```

```
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

instanceof-avsmalning

`instanceof`-operatoravsmalning i TypeScript är ett sätt att avsmalma typen av en variabel baserat på dess konstruktorfunktion, genom att kontrollera om ett objekt är en instans av en viss klass eller gränssnitt.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

Tilldelningar

TypeScript-avsmalning med hjälp av tilldelningar är ett sätt att avsmalma typen av en variabel baserat på det tilldelade värdet. När en variabel tilldelas ett värde härleder TypeScript dess typ baserat på det tilldelade värdet, och avsmalmar variabelns typ för att matcha den härledda typen.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

Kontrollflödesanalys

Kontrollflödesanalys i TypeScript är ett sätt att statiskt analysera kodflödet för att härleda typer av variabler, vilket gör det möjligt för kompilatorn att avsmalma typerna av dessa variabler efter behov, baserat på resultaten av analysen.

Före TypeScript 4.4 tillämpades kodflödesanalys enbart på kod inom en if-sats, men från och med TypeScript 4.4 kan den även tillämpas på villkorliga uttryck och diskriminantegenskapsåtkomster som indirekt refereras genom const-variabler.

Till exempel:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

Några exempel där avsmalning inte sker:

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
          x.length; // Fel, ingen insnävning eftersom isString
inte är const
    }
};

const f6 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
              obj.foo; // Fel, ingen insnävning eftersom obj
tilldelas i funktionskroppen
```

```
    }
};
```

Observera: Upp till fem nivåer av indirektion analyseras i villkorliga uttryck.

Typpredikat

Typpredikat i TypeScript är funktioner som returnerar ett booleskt värde och används för att avsmalma typen av en variabel till en mer specifik typ.

```
const isString = (value: unknown): value is string => typeof
value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

TypeScript 5.5 härleder automatiskt typpredikat (som `x is T`) i funktioner som `.filter`, så att den vet när värden som undefined tas bort – vilket ger mer exakta typer och färre fel; detta fungerar för tydliga kontroller (t.ex. `x !== undefined`) men inte tvetydiga som `!!x`.

```
const nums = [1, null, 2].filter(x => x !== null);
```

Diskriminerade unioner

Diskriminerade unioner i TypeScript är en typ av unionstyp som använder en gemensam egenskap, känd som

diskriminanten, för att avsmalma uppsättningen av möjliga typer för unionen.

```
type Square = {
    kind: 'square'; // Diskriminant
    size: number;
};

type Circle = {
    kind: 'circle'; // Diskriminant
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

Never-typen

När en variabel avsmalnas till en typ som inte kan innehålla några värden, kommer TypeScript-kompilatorn att härleda att variabeln måste vara av typen `never`. Detta beror på att never-typen representerar ett värde som aldrig kan produceras.

```
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
        // val har typen never här eftersom det aldrig kan vara något annat än en sträng eller ett tal
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

Uttömmande kontroll

Uttömmande kontroll är en funktion i TypeScript som säkerställer att alla möjliga fall av en diskriminerad union hanteras i en `switch`-sats eller en `if`-sats.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
                console.log(exhaustiveCheck); // Den här raden kommer aldrig att köras
    }
};
```

`never`-typen används för att säkerställa att default-fallet är uttömmande och att TypeScript kommer att ge ett fel om ett nytt värde läggs till i Direction-typen utan att det hanteras i switch-satsen.

Objekttyper

I TypeScript beskriver objekttyper formen på ett objekt. De specificerar namnen och typerna på objektets egenskaper, samt huruvida dessa egenskaper är obligatoriska eller valfria.

I TypeScript kan du definiera objekttyper på två primära sätt:

Interface som definierar formen på ett objekt genom att specificera namnen, typerna och valfrihet hos dess egenskaper.

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

Typalias, liknande ett interface, definierar formen på ett objekt. Det kan dock även skapa en ny anpassad typ som baseras på en befintlig typ eller en kombination av befintliga typer. Detta inkluderar att definiera unionstyper, intersektionstyper och andra komplexa typer.

```
type Point = {
    x: number;
    y: number;
};
```

Det är också möjligt att definiera en typ anonymt:

```ts
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

Tuppeltyp (Anonym)

En tuppeltyp är en typ som representerar en array med ett fast antal element och deras motsvarande typer. En tuppeltyp tvingar fram ett specifikt antal element och deras respektive typer i en fast ordning. Tuppeltyper är användbara när du vill representera en samling värden med specifika typer, där positionen för varje element i arrayen har en specifik betydelse.

```ts
type Point = [number, number];
```

Namngiven tuppeltyp (Märkt)

Tuppeltyper kan inkludera valfria etiketter eller namn för varje element. Dessa etiketter är till för läsbarhet och verktygsstöd, och påverkar inte de operationer du kan utföra med dem.

```ts
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Namngiven tupel plus anonym tupel
```

Tuppel med fast längd

En tuppel med fast längd är en specifik typ av tuppel som tvingar fram ett fast antal element av specifika typer, och tillåter inte några ändringar av tuppelns längd efter att den har definierats.

Tupplar med fast längd är användbara när du behöver representera en samling värden med ett specifikt antal element

och specifika typer, och du vill säkerställa att längden och typerna av tuppeln inte kan ändras oavsiktligt.

```
const x = [10, 'hello'] as const;
x.push(2); // Fel
```

Unionstyp

En unionstyp är en typ som representerar ett värde som kan vara en av flera typer. Unionstyper betecknas med symbolen | mellan varje möjlig typ.

```
let x: string | number;
x = 'hello'; // Giltigt
x = 123; // Giltigt
```

Intersektionstyper

En intersektionstyp är en typ som representerar ett värde som har alla egenskaper hos två eller flera typer. Intersektionstyper betecknas med symbolen & mellan varje typ.

```
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Snitt

const j: J = {
    a: 'a',
    b: 'b',
};
```

Typindexering

Typindexering avser möjligheten att definiera typer som kan indexeras med en nyckel som inte är känd i förväg, genom att använda en indexsignatur för att specificera typen för egenskaper som inte uttryckligen deklarerats.

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Returnerar a
```

Typ från värde

Typ från värde i TypeScript avser den automatiska härledningen av en typ från ett värde eller uttryck genom typinferens.

```
const x = 'x'; // TypeScript härleder 'x' som en strängliteral med 'const' (oföränderlig), men breddar den till 'string' med 'let' (kan tilldelas på nytt).
```

Typ från funktionsreturvärde

Typ från funktionsreturvärde avser möjligheten att automatiskt härleda returtypen av en funktion baserat på dess implementation. Detta gör att TypeScript kan bestämma typen av det värde som returneras av funktionen utan explicita typannoteringar.

```
const add = (x: number, y: number) => x + y; // TypeScript kan härleda att funktionens returtyp är number
```

Typ från modul

Typ från modul avser möjligheten att använda en moduls exporterade värden för att automatiskt härleda deras typer. När en modul exporterar ett värde med en specifik typ kan TypeScript använda den informationen för att automatiskt härleda typen av det värdet när det importeras till en annan modul.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r har typen number
```

Mappade typer

Mappade typer i TypeScript låter dig skapa nya typer baserade på en befintlig typ genom att transformera varje egenskap med hjälp av en mappningsfunktion. Genom att mappa befintliga typer kan du skapa nya typer som representerar samma information i ett annat format. För att skapa en mappad typ kommer du åt egenskaperna hos en befintlig typ med `keyof`-operatorn och ändrar dem sedan för att producera en ny typ. I följande exempel:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
```

```
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

definierar vi MyMappedType för att mappa över T:s egenskaper och skapa en ny typ där varje egenskap är en array av sin ursprungliga typ. Med hjälp av detta skapar vi MyNewType för att representera samma information som MyType, men med varje egenskap som en array.

Modifierare för mappade typer

Modifierare för mappade typer i TypeScript möjliggör transformation av egenskaper inom en befintlig typ:

- `readonly` eller `+readonly`: Detta gör en egenskap i den mappade typen skrivskyddad.
- `-readonly`: Detta gör att en egenskap i den mappade typen kan ändras.
- `?`: Detta gör en egenskap i den mappade typen valfri.

Exempel:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // Alla egenskaper markerade som skrivskyddade

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // Alla egenskaper markerade som föränderliga

type MyPartial<T> = { [P in keyof T]?: T[P] }; // Alla egenskaper markerade som valfria
```

Villkorliga typer

Villkorliga typer är ett sätt att skapa en typ som beror på ett villkor, där typen som ska skapas bestäms baserat på resultatet av villkoret. De definieras med nyckelordet `extends` och en ternär operator för att villkorligt välja mellan två typer.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // Typen true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Typen false
```

Distributiva villkorliga typer

Distributiva villkorliga typer är en funktion som gör det möjligt att distribuera en typ över en union av typer, genom att tillämpa en transformation på varje medlem av unionen individuellt. Detta kan vara särskilt användbart vid arbete med mappade typer eller typer av högre ordning.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number | boolean | null
```

infer-typinferens i villkorliga typer

Nyckelordet `infer` används i villkorliga typer för att härleda (extrahera) typen av en generisk parameter från en typ som beror på den. Detta gör att du kan skriva mer flexibla och återanvändbara typdefinitioner.

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

Fördefinierade villkorliga typer

I TypeScript är fördefinierade villkorliga typer inbyggda villkorliga typer som tillhandahålls av språket. De är utformade för att utföra vanliga typtransformationer baserade på egenskaperna hos en given typ.

`Exclude<UnionType, ExcludedType>`: Denna typ tar bort alla typer från Type som kan tilldelas till ExcludedType.

`Extract<Type, Union>`: Denna typ extraherar alla typer från Union som kan tilldelas till Type.

`NonNullable<Type>`: Denna typ tar bort null och undefined från Type.

`ReturnType<Type>`: Denna typ extraherar returtypen av en funktionstyp Type.

`Parameters<Type>`: Denna typ extraherar parametertyperna av en funktionstyp Type.

`Required<Type>`: Denna typ gör alla egenskaper i Type obligatoriska.

`Partial<Type>`: Denna typ gör alla egenskaper i Type valfria.

`Readonly<Type>`: Denna typ gör alla egenskaper i Type skrivskyddade.

Mall-unionstyper

Mall-unionstyper kan användas för att slå samman och manipulera text inuti typsystemet, till exempel:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active" | "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Any-typen

any-typen är en speciell typ (universell supertyp) som kan användas för att representera vilken typ av värde som helst (primitiver, objekt, arrayer, funktioner, fel, symboler). Den används ofta i situationer där typen av ett värde inte är känd vid kompilering, eller vid arbete med värden från externa API:er eller bibliotek som inte har TypeScript-typningar.

Genom att använda any-typen indikerar du för TypeScript-kompilatorn att värden ska representeras utan några begränsningar. För att maximera typsäkerheten i din kod, överväg följande:

- Begränsa användningen av any till specifika fall där typen verkligen är okänd.
- Returnera inte any-typer från en funktion, eftersom detta försvagar typsäkerheten i kod som använder den.
- Istället för any, använd `@ts-ignore` om du behöver tysta kompilatorn.

```
let value: any;
value = true; // Giltigt
value = 7; // Giltigt
```

Unknown-typen

I TypeScript representerar unknown-typen ett värde av en okänd typ. Till skillnad från any-typen, som tillåter vilken typ av värde som helst, kräver unknown en typkontroll eller assertion innan den kan användas på ett specifikt sätt, så inga operationer är tillåtna på en unknown utan att först göra en assertion eller avsmalning till en mer specifik typ.

unknown-typen kan bara tilldelas till any-typen och unknown-typen själv, den är ett typsäkert alternativ till any.

```
let value: unknown;

let value1: unknown = value; // Giltigt
let value2: any = value; // Giltigt
let value3: boolean = value; // Ogiltigt
let value4: number = value; // Ogiltigt

const add = (a: unknown, b: unknown): number | undefined =>
     typeof a === 'number' && typeof b === 'number' ? a + b :
undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Void-typen

Typen void används för att indikera att en funktion inte returnerar något värde.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

Användning av Never-typen

Typen `never` representerar värden som aldrig förekommer. Den används för att beteckna funktioner eller uttryck som aldrig returnerar eller kastar ett fel.

Till exempel en oändlig loop:

```
const infiniteLoop = (): never => {
    while (true) {
        // gör något
    }
};
```

Kasta ett fel:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

Typen `never` är användbar för att säkerställa typsäkerhet och fånga potentiella fel i din kod. Den hjälper TypeScript att analysera och härleda mer precisa typer när den används i kombination med andra typer och kontrollflödessatser, till exempel:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // flytta upp
            break;
        case 'down':
            // flytta ned
            break;
        default:
            const exhaustiveCheck: never = direction;
                        throw new Error(`Unhandled direction:
${exhaustiveCheck}`);
```

```
    }
};
```

Interface och Type

Gemensam syntax

I TypeScript definierar interface strukturen hos objekt och specificerar namnen och typerna på egenskaper eller metoder som ett objekt måste ha. Den gemensamma syntaxen för att definiera ett interface i TypeScript är följande:

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

På liknande sätt för typdefinition:

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` eller `type TypeName`: Definierar namnet på interfacet. `property1: Type1`: Specificerar interfacets egenskaper tillsammans med deras motsvarande typer. Flera egenskaper kan definieras, var och en separerad med ett semikolon. `method1(arg1: ArgType1, arg2: ArgType2): ReturnType;`: Specificerar interfacets metoder. Metoder definieras med sina namn, följt av en parameterlista inom

parenteser och returtypen. Flera metoder kan definieras, var och en separerad med ett semikolon.

Exempel på interface:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

Exempel på type:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

I TypeScript används typer för att definiera formen på data och upprätthålla typkontroll. Det finns flera vanliga syntaxer för att definiera typer i TypeScript, beroende på det specifika användningsfallet. Här är några exempel:

Grundläggande typer

```
let myNumber: number = 123; // typen number
let myBoolean: boolean = true; // typen boolean
let myArray: string[] = ['a', 'b']; // array av strängar
let myTuple: [string, number] = ['a', 123]; // tupel
```

Objekt och Interface

```
const x: { name: string; age: number } = { name: 'Simon', age:
7 };
```

Union- och Intersection-typer

```
type MyType = string | number; // Unionstyp
let myUnion: MyType = 'hello'; // Kan vara en sträng
myUnion = 123; // Eller ett tal

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Snittyp
let myCombined: CombinedType = { name: 'John', age: 25 }; //
Objekt med både egenskaperna name och age
```

Inbyggda primitiva typer

TypeScript har flera inbyggda primitiva typer som kan användas för att definiera variabler, funktionsparametrar och returtyper:

- `number`: Representerar numeriska värden, inklusive heltal och flyttal.
- `string`: Representerar textdata.
- `boolean`: Representerar logiska värden, som kan vara antingen true eller false.
- `null`: Representerar frånvaron av ett värde.
- `undefined`: Representerar ett värde som inte har tilldelats eller inte har definierats.
- `symbol`: Representerar en unik identifierare. Symboler används vanligtvis som nycklar för objektegenskaper.
- `bigint`: Representerar heltal med godtycklig precision.
- `any`: Representerar en dynamisk eller okänd typ. Variabler av typen any kan innehålla värden av vilken typ som helst, och de kringgår typkontroll.
- `void`: Representerar frånvaron av någon typ. Den används vanligtvis som returtyp för funktioner som inte returnerar ett värde.

- `never`: Representerar en typ för värden som aldrig förekommer. Den används vanligtvis som returtyp för funktioner som kastar ett fel eller går in i en oändlig loop.

Vanliga inbyggda JS-objekt

TypeScript är en utökning av JavaScript och inkluderar alla vanligt använda inbyggda JavaScript-objekt. Du kan hitta en utförlig lista över dessa objekt på Mozilla Developer Networks (MDN) dokumentationswebbplats: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Här är en lista över några vanligt använda inbyggda JavaScript-objekt:

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map
- Set
- Promise
- Intl

Överlagringar

Funktionsöverlagringar i TypeScript låter dig definiera flera funktionssignaturer för ett enda funktionsnamn, vilket gör det möjligt att definiera funktioner som kan anropas på flera sätt. Här är ett exempel:

```
// Överlagringar
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Implementation
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Giltigt
sayHi(['aa', 'bb']); // Giltigt
```

Här är ytterligare ett exempel på användning av funktionsöverlagringar inom en `class`:

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }

    // överlagring
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;
```

```
    // implementation
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
                    return  name.map(name  =>  `${this.message},
${name}!`);
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

Sammanslagning och utökning

Sammanslagning och utökning avser två olika koncept relaterade till att arbeta med typer och interface.

Sammanslagning låter dig kombinera flera deklarationer med samma namn till en enda definition, till exempel när du definierar ett interface med samma namn flera gånger:

```
interface X {
    a: string;
}

interface X {
    b: number;
}

const person: X = {
    a: 'a',
    b: 7,
};
```

Utökning avser möjligheten att utöka eller ärva från befintliga typer eller interface för att skapa nya. Det är en mekanism för att lägga till ytterligare egenskaper eller metoder till en befintlig typ utan att ändra dess ursprungliga definition. Exempel:

```
interface Animal {
    name: string;
    eat(): void;
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

Skillnader mellan Type och Interface

Deklarationssammanslagning (augmentering):

Interface stöder deklarationssammanslagning, vilket innebär att du kan definiera flera interface med samma namn, och TypeScript kommer att slå samman dem till ett enda interface med de kombinerade egenskaperna och metoderna. Å andra sidan stöder typer inte deklarationssammanslagning. Detta kan vara användbart när du vill lägga till extra funktionalitet eller

anpassa befintliga typer utan att ändra de ursprungliga definitionerna eller korrigera saknade eller felaktiga typer.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

Utökning av andra typer/interface:

Både typer och interface kan utöka andra typer/interface, men syntaxen är annorlunda. Med interface använder du nyckelordet `extends` för att ärva egenskaper och metoder från andra interface. Ett interface kan dock inte utöka en komplex typ som en union-typ.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
    y: 123,
    z: 'z',
};
```

För typer använder du operatorn & för att kombinera flera typer till en enda typ (intersection).

```ts
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

Union- och Intersection-typer:

Typer är mer flexibla när det gäller att definiera union- och intersection-typer. Med nyckelordet `type` kan du enkelt skapa union-typer med operatorn `|` och intersection-typer med operatorn `&`. Även om interface också kan representera union-typer indirekt, har de inget inbyggt stöd för intersection-typer.

```ts
type Department = 'dep-x' | 'dep-y'; // Union

type Person = {
    name: string;
    age: number;
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Snitt
```

Exempel med interface:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // Union av gränssnitt
```

Klass

Vanlig klasssyntax

Nyckelordet `class` används i TypeScript för att definiera en klass. Nedan kan du se ett exempel:

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
```

Nyckelordet `class` används för att definiera en klass som heter “Person”.

Klassen har två privata egenskaper: name av typen `string` och age av typen `number`.

Konstruktorn definieras med nyckelordet `constructor`. Den tar name och age som parametrar och tilldelar dem till motsvarande egenskaper.

Klassen har en `public` metod som heter sayHi som loggar ett hälsningsmeddelande.

För att skapa en instans av en klass i TypeScript kan du använda nyckelordet `new` följt av klassnamnet, följt av parenteser `()`. Till exempel:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Utdata: Hello, my name is John Doe and I
am 25 years old.
```

Konstruktor

Konstruktorer är speciella metoder inom en klass som används för att initiera objektets egenskaper när en instans av klassen skapas.

```
class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
```

```
            `Hello, my name is ${this.name} and I'm ${this.age}
years old.`
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

Det är möjligt att överlagra en konstruktor med följande syntax:

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

I TypeScript är det möjligt att definiera flera konstruktoröverlagringar, men du kan bara ha en implementering som måste vara kompatibel med alla överlagringar. Detta kan uppnås genom att använda en valfri parameter.

```
class Person {
    name: string;
    age: number;
```

```
    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

Privata och skyddade konstruktorer

I TypeScript kan konstruktorer markeras som privata eller skyddade, vilket begränsar deras åtkomlighet och användning.

Privata konstruktorer: Kan bara anropas inom själva klassen. Privata konstruktorer används ofta i scenarier där du vill upprätthålla ett singleton-mönster eller begränsa skapandet av instanser till en fabriksmetod inom klassen.

Skyddade konstruktorer: Skyddade konstruktorer är användbara när du vill skapa en basklass som inte ska instansieras direkt men kan utökas av underklasser.

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// Ett försök att instansiera basklassen direkt ger ett fel
// const baseObj = new BaseClass(); // Fel: Konstruktorn för
klassen 'BaseClass' är skyddad.

// Skapa en instans av den härledda klassen
const derivedObj = new DerivedClass(10);
```

Åtkomstmodifierare

Åtkomstmodifierarna `private`, `protected` och `public` används för att styra synligheten och åtkomsten till klassmedlemmar, såsom egenskaper och metoder, i TypeScript-klasser. Dessa modifierare är viktiga för att upprätthålla inkapsling och för att etablera gränser för åtkomst och modifiering av en klass interna tillstånd.

Modifieraren `private` begränsar åtkomsten till klassmedlemmen enbart inom den innehållande klassen.

Modifieraren `protected` tillåter åtkomst till klassmedlemmen inom den innehållande klassen och dess härledda klasser.

Modifieraren `public` ger obegränsad åtkomst till klassmedlemmen och tillåter att den nås från var som helst.

Get och Set

Getters och setters är speciella metoder som låter dig definiera anpassat åtkomst- och ändringsbeteende för klassegenskaper. De gör det möjligt att kapsla in det interna tillståndet hos ett objekt och tillhandahålla ytterligare logik vid hämtning eller inställning av egenskapsvärden. I TypeScript definieras getters och setters med nyckelorden `get` respektive `set`. Här är ett exempel:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

Auto-accessorer i klasser

TypeScript version 4.9 lägger till stöd för auto-accessorer, en kommande ECMAScript-funktion. De liknar klassegenskaper men deklareras med nyckelordet “accessor”.

```
class Animal {
    accessor name: string;
```

```
    constructor(name: string) {
        this.name = name;
    }
}
```

Auto-accessorer "avsockras" till privata `get`- och `set`-accessorer som opererar på en otillgänglig egenskap.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

this

I TypeScript refererar nyckelordet `this` till den aktuella instansen av en klass inom dess metoder eller konstruktorer. Det ger dig möjlighet att komma åt och modifiera klassens egenskaper och metoder inifrån dess eget scope. Det erbjuder ett sätt att komma åt och manipulera det interna tillståndet hos ett objekt inom dess egna metoder.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
```

```
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

Parameteregenskaper

Parameteregenskaper gör det möjligt att deklarera och initiera klassegenskaper direkt i konstruktorns parametrar, vilket undviker överflödig kod. Exempel:

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
        // Nyckelorden "private" och "public" i konstruktorn
           // deklarerar och initierar automatiskt motsvarande
klassegenskaper.
    }
    public introduce(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

Abstrakta klasser

Abstrakta klasser används i TypeScript främst för arv; de erbjuder ett sätt att definiera gemensamma egenskaper och metoder som kan ärvas av underklasser. Detta är användbart när du vill definiera gemensamt beteende och säkerställa att underklasser implementerar vissa metoder. De ger ett sätt att skapa en hierarki av klasser där den abstrakta basklassen tillhandahåller ett delat gränssnitt och gemensam funktionalitet för underklasserna.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Utdata: Whiskers meows.
```

Med generics

Klasser med generics gör det möjligt att definiera återanvändbara klasser som kan arbeta med olika typer.

```
class Container<T> {
    private item: T;
```

```
    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

Dekoratörer

Dekoratörer tillhandahåller en mekanism för att lägga till metadata, ändra beteende, validera eller utöka funktionaliteten hos målelementet. De är funktioner som körs vid körning. Flera dekoratörer kan tillämpas på en deklaration.

Dekoratörer är experimentella funktioner, och följande exempel är bara kompatibla med TypeScript version 5 eller senare med ES6.

För TypeScript-versioner före 5 bör de aktiveras med egenskapen `experimentalDecorators` i din `tsconfig.json` eller genom att använda `--experimentalDecorators` på kommandoraden (men följande exempel kommer inte att fungera).

Några vanliga användningsfall för dekoratörer inkluderar:

- Övervakning av egenskapsändringar.
- Övervakning av metodanrop.
- Tillägg av extra egenskaper eller metoder.
- Validering vid körning.
- Automatisk serialisering och deserialisering.
- Loggning.
- Auktorisering och autentisering.
- Felhantering.

Observera: Dekoratörer för version 5 tillåter inte dekorering av parametrar.

Typer av dekoratörer:

Klassdekoratörer

Klassdekoratörer är användbara för att utöka en befintlig klass, till exempel genom att lägga till egenskaper eller metoder, eller samla instanser av en klass. I följande exempel lägger vi till en `toString`-metod som konverterar klassen till en strängrepresentation.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
```

```
        }
    };
}

@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* Skriver ut:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

Egenskapsdekoratör

Egenskapsdekoratörer är användbara för att ändra beteendet hos en egenskap, till exempel genom att ändra initieringsvärden. I följande kod har vi ett skript som ställer in en egenskap till att alltid vara i versaler:

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}
```

```
class MyClass {
    @upperCase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // Skriver ut: HELLO!
```

Metoddekoratör

Metoddekoratörer låter dig ändra eller förbättra beteendet hos metoder. Nedan följer ett exempel på en enkel logger:

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

       function replacementMethod(this: This, ...args: Args):
Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
```

```
    }
}

new MyClass().sayHello();
```

Det loggar:

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

Getter- och setter-dekoratörer

Getter- och setter-dekoratörer låter dig ändra eller förbättra beteendet hos klass-accessorer. De är användbara, till exempel, för validering av egenskapstilldelningar. Här är ett enkelt exempel på en getter-dekoratör:

```
function range<This, Return extends number>(min: number, max:
number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
}
```

```
class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Giltigt: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // Kastar undantag: Invalid!
```

Dekoratörmetadata

Dekoratörmetadata förenklar processen för dekoratörer att tillämpa och använda metadata i valfri klass. De kan komma åt en ny metadataegenskap på kontextobjektet, som kan fungera som en nyckel för både primitiva värden och objekt. Metadatainformation kan nås på klassen via `Symbol.metadata`.

Metadata kan användas för olika ändamål, till exempel felsökning, serialisering eller beroendeinjektion med dekoratörer.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Enkel
polyfill

type Context =
```

```
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
      | ClassMethodDecoratorContext; // Kontexten innehåller
egenskapsmetadata: DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // Ange metadataobjektet med ett primitivt värde
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Hämta
metadatainformation

console.log(JSON.stringify(metadata)); //
{"bar":true,"baz":true,"foo":true}
```

Arv

Arv avser mekanismen genom vilken en klass kan ärva egenskaper och metoder från en annan klass, känd som basklassen eller superklassen. Den härledda klassen, även kallad barnklassen eller underklassen, kan utöka och specialisera basklassens funktionalitet genom att lägga till nya egenskaper och metoder eller åsidosätta befintliga.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Skapa en instans av basklassen
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Skapa en instans av den härledda klassen
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript stöder inte multipelt arv i traditionell bemärkelse utan tillåter istället arv från en enda basklass. TypeScript stöder flera gränssnitt. Ett gränssnitt kan definiera ett kontrakt för strukturen hos ett objekt, och en klass kan implementera flera

gränssnitt. Detta gör det möjligt för en klass att ärva beteende och struktur från flera källor.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

Nyckelordet `class` i TypeScript, liknande JavaScript, kallas ofta för syntaktiskt socker. Det introducerades i ECMAScript 2015 (ES6) för att erbjuda en mer välbekant syntax för att skapa och arbeta med objekt på ett klassbaserat sätt. Det är dock viktigt att notera att TypeScript, som en utökning av JavaScript, slutligen kompileras ner till JavaScript, som i grunden förblir prototypbaserat.

Statiska medlemmar

TypeScript har statiska medlemmar. För att komma åt de statiska medlemmarna i en klass kan du använda klassnamnet följt av en punkt, utan att behöva skapa ett objekt.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

Egenskapsinitiering

Det finns flera sätt att initiera egenskaper för en klass i TypeScript:

Inline:

I följande exempel kommer dessa initiala värden att användas när en instans av klassen skapas.

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

I konstruktorn:

```
class MyClass {
    property1: string;
    property2: number;
```

```
    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

Använda konstruktorparametrar:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
            // Värdena behöver inte tilldelas egenskaperna
uttryckligen.
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

Metodöverlagring

Metodöverlagring gör det möjligt för en klass att ha flera metoder med samma namn men olika parametertyper eller ett annat antal parametrar. Detta gör att vi kan anropa en metod på olika sätt baserat på de argument som skickas.

```
class MyClass {
    add(a: number, b: number): number; // Överlagringssignatur
1
    add(a: string, b: string): string; // Överlagringssignatur
2
```

```
      add(a: number | string, b: number | string): number |
string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}

const r = new MyClass();
console.log(r.add(10, 5)); // Skriver ut 15
```

Generics

Generics gör det möjligt att skapa återanvändbara komponenter och funktioner som kan arbeta med flera typer. Med generics kan du parameterisera typer, funktioner och gränssnitt, vilket gör att de kan arbeta med olika typer utan att explicit specificera dem i förväg.

Generics gör det möjligt att göra koden mer flexibel och återanvändbar.

Generisk typ

För att definiera en generisk typ använder du vinkelparenteser (<>) för att specificera typparametrarna, till exempel:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);
```

```
const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

Generiska klasser

Generics kan även tillämpas på klasser, på så sätt kan de arbeta med flera typer genom att använda typparametrar. Detta är användbart för att skapa återanvändbara klassdefinitioner som kan arbeta med olika datatyper samtidigt som typsäkerheten bibehålls.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

Generiska begränsningar

Generiska parametrar kan begränsas med nyckelordet `extends` följt av en typ eller ett gränssnitt som typparametern måste uppfylla.

I följande exempel måste T innehålla en egenskap `length` för att vara giltig:

```
const printLen = <T extends { length: number }>(value: T): void
=> {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // Ogiltigt
```

En intressant funktion hos generics som introducerades i version 3.4 RC är typinferens för högre ordningens funktioner, vilket introducerade propagerade generiska typargument:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Denna funktionalitet möjliggör enklare typsäker punktfri programmering (pointfree style) vilket är vanligt inom funktionell programmering.

Generisk kontextuell avsmalning

Kontextuell avsmalning för generics är mekanismen i TypeScript som gör det möjligt för kompilatorn att smalna av

typen för en generisk parameter baserat på det sammanhang där den används. Det är användbart vid arbete med generiska typer i villkorssatser:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Värdet snävas in till typen 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // Värdet snävas in till typen 'number'
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

Raderade strukturella typer

I TypeScript behöver objekt inte matcha en specifik, exakt typ. Till exempel, om vi skapar ett objekt som uppfyller ett gränssnitts krav, kan vi använda det objektet på platser där det gränssnittet krävs, även om det inte finns någon explicit koppling mellan dem. Exempel:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
```

```
};

log(obj); // Giltigt
```

Namnrymder

I TypeScript används namnrymder (namespaces) för att organisera kod i logiska behållare, förhindra namnkollisioner och ge ett sätt att gruppera relaterad kod tillsammans. Användningen av nyckelordet `export` tillåter åtkomst till namnrymden i "utomstående" moduler.

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

Symboler

Symboler är en primitiv datatyp som representerar ett oföränderligt värde som garanterat är globalt unikt under hela programmets livstid.

Symboler kan användas som nycklar för objektegenskaper och ger ett sätt att skapa icke-uppräkningsbara egenskaper.

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

I WeakMaps och WeakSets är symboler nu tillåtna som nycklar.

Trippelsnedstreck-direktiv

Trippelsnedstreck-direktiv är speciella kommentarer som ger instruktioner till kompilatorn om hur en fil ska bearbetas. Dessa direktiv börjar med tre på varandra följande snedstreck (`///`) och placeras vanligtvis överst i en TypeScript-fil och har ingen effekt på körbeteendet.

Trippelsnedstreck-direktiv används för att referera till externa beroenden, specificera modulinläsningsbeteende, aktivera/inaktivera vissa kompilatorfunktioner, med mera. Några exempel:

Referera till en deklarationsfil:

```
/// <reference path="path/to/declaration/file.d.ts" />
```

Ange modulformat:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Aktivera kompilatoralternativ, i följande exempel strikt läge:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

Typmanipulation

Skapa typer från typer

Det är möjligt att skapa nya typer genom att komponera, manipulera eller transformera befintliga typer.

Intersektionstyper (`&`):

Gör det möjligt att kombinera flera typer till en enda typ:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Snitt av A och B
const obj: C = { foo: 42, bar: 'hello' };
```

Unionstyper (`|`):

Gör det möjligt att definiera en typ som kan vara en av flera typer:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Mappade typer:

Gör det möjligt att transformera egenskaperna hos en befintlig typ för att skapa en ny typ:

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
```

```
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Egenskaperna blir
skrivskyddade
```

Villkorliga typer:

Gör det möjligt att skapa typer baserat på vissa villkor:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P :
never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

Indexerade åtkomsttyper

I TypeScript är det möjligt att komma åt och manipulera typerna av egenskaper inom en annan typ med hjälp av ett index, `Type[Key]`.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

Verktygstyper

Flera inbyggda verktygstyper kan användas för att manipulera typer. Nedan följer en lista över de mest använda:

Awaited<T>

Konstruerar en typ som rekursivt packar upp Promise-typer.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

Konstruerar en typ där alla egenskaper i T är satta som valfria.

```
type Person = {
    name: string;
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?:
number | undefined; }
```

Required<T>

Konstruerar en typ där alla egenskaper i T är satta som obligatoriska.

```
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

Konstruerar en typ där alla egenskaper i T är satta som skrivskyddade.

```
type Person = {
    name: string;
    age: number;
```

```
};

type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Ogiltigt
```

Record<K, T>

Konstruerar en typ med en uppsättning egenskaper K av typen T.

```
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

Konstruerar en typ genom att välja ut de angivna egenskaperna K från T.

```
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

Konstruerar en typ genom att utelämna de angivna egenskaperna K från T.

```
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

Konstruerar en typ genom att exkludera alla värden av typen U från T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

Konstruerar en typ genom att extrahera alla värden av typen U från T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

Konstruerar en typ genom att exkludera null och undefined från T.

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

Extraherar parametertyperna för en funktionstyp T.

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

Extraherar parametertyperna för en konstruktorfunktionstyp T.

```
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type  PersonConstructorParams  =  ConstructorParameters<typeof
Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

Extraherar returtypen för en funktionstyp T.

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

Extraherar instanstypen för en klasstyp T.

```
class Person {
    name: string;
```

```
    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}

type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

Extraherar typen av ‘this’-parametern från en funktionstyp T.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type  PersonThisType  =  ThisParameterType<Person['greet']>;  //
Person
```

OmitThisParameter<T>

Tar bort ‘this’-parametern från en funktionstyp T.

```
function capitalize(this: String) {
                        return      this[0].toUpperCase()      +
this.substring(1).toLowerCase();
}
```

```
type CapitalizeType = OmitThisParameter<typeof capitalize>; // () => string
```

ThisType<T>

Fungerar som en markör för en kontextuell `this`-typ.

```
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } & ThisType<Logger> = {
    hello: function () {
        this.log('some error'); // Giltigt eftersom "log" är en del av "this".
        this.update(); // Ogiltigt
    },
};
```

Uppercase<T>

Gör namnet på indatatypen T till versaler.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

Gör namnet på indatatypen T till gemener.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

Gör första bokstaven i namnet på indatatypen T till versal.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

Gör första bokstaven i namnet på indatatypen T till gemen.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer är en verktygstyp utformad för att blockera automatisk typinferens inom ramen för en generisk funktion.

Exempel:

```
// Automatisk typhärledning inom en generisk funktions räckvidd.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Typen här är ("a" | "b" | "c")[]
```

Med NoInfer:

```
// Exempelfunktion som använder NoInfer för att förhindra typhärledning
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Fel: Ett argument av typen '"c"' kan inte tilldelas en parameter av typen '"a" | "b"'.
```

Övrigt

Fel och undantagshantering

TypeScript låter dig fånga och hantera fel med hjälp av JavaScripts standardmekanismer för felhantering:

Try-Catch-Finally-block:

```
try {
    // Kod som kan kasta ett fel
} catch (error) {
    // Hantera felet
} finally {
    // Kod som alltid körs, finally är valfritt
}
```

Du kan också hantera olika typer av fel:

```
try {
    // Kod som kan kasta olika typer av fel
} catch (error) {
    if (error instanceof TypeError) {
        // Hantera TypeError
    } else if (error instanceof RangeError) {
        // Hantera RangeError
    } else {
        // Hantera andra fel
    }
}
```

Anpassade feltyper:

Det är möjligt att specificera mer specifika fel genom att utöka `Error`-klassen:

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
```

```
    }
}

throw new CustomError('This is a custom error.');
```

Mixin-klasser

Mixin-klasser låter dig kombinera och komponera beteende från flera klasser till en enda klass. De erbjuder ett sätt att återanvända och utöka funktionalitet utan behov av djupa arvskedjor.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Utöka MyClass med beteendet från Identifiable och Selectable
interface MyClass extends Identifiable, Selectable {}

// Funktion för att tillämpa mixins på en klass
```

```
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {

Object.getOwnPropertyNames(baseCtor.prototype).forEach(name   =>
{
            let descriptor = Object.getOwnPropertyDescriptor(
                baseCtor.prototype,
                name
            );
            if (descriptor) {
                  Object.defineProperty(source.prototype, name,
descriptor);
            }
        });
    });
}

// Tillämpa mixins på MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

Asynkrona språkfunktioner

Eftersom TypeScript är en superset av JavaScript har det inbyggda asynkrona språkfunktioner från JavaScript som:

Promises:

Promises är ett sätt att hantera asynkrona operationer och deras resultat med hjälp av metoder som `.then()` och `.catch()` för att hantera framgångs- och feltillstånd.

Läs mer: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

Nyckelorden async/await erbjuder en mer synkront utseende syntax för att arbeta med Promises. Nyckelordet `async` används för att definiera en asynkron funktion, och nyckelordet `await` används inom en async-funktion för att pausa körningen tills ett Promise har lösts eller avvisats.

Läs mer: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

Följande API:er stöds väl i TypeScript:

Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Iteratorer och generatorer

Både iteratorer och generatorer stöds väl i TypeScript.

Iteratorer är objekt som implementerar iteratorprotokollet och erbjuder ett sätt att komma åt element i en samling eller sekvens ett i taget. Det är en struktur som innehåller en pekare

till nästa element i iterationen. De har en `next()`-metod som returnerar nästa värde i sekvensen tillsammans med en boolean som anger om sekvensen är `done` (klar).

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

Generatorer är speciella funktioner definierade med syntaxen `function*` som förenklar skapandet av iteratorer. De använder

nyckelordet `yield` för att definiera sekvensen av värden och pausar och återupptar automatiskt körningen när värden efterfrågas.

Generatorer gör det enklare att skapa iteratorer och är särskilt användbara för att arbeta med stora eller oändliga sekvenser.

Exempel:

```
function* numberGenerator(start: number, end: number): Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript stöder också asynkrona iteratorer och asynkrona generatorer.

Läs mer:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

TsDocs JSDoc-referens

När man arbetar med en JavaScript-kodbas är det möjligt att hjälpa TypeScript att härleda rätt typ genom att använda JSDoc-kommentarer med ytterligare annotationer för att ge typinformation.

Exempel:

```
/**
 * Beräknar potensen av ett givet tal
 * @constructor
 * @param {number} base - Uttryckets basvärde
 * @param {number} exponent - Uttryckets exponentvärde
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number): number
```

Fullständig dokumentation finns på denna länk: https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

Från version 3.7 är det möjligt att generera .d.ts-typdefinitioner från JavaScript JSDoc-syntax. Mer information finns här: https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

Paket under @types-organisationen är speciella paketnamnkonventioner som används för att tillhandahålla typdefinitioner för befintliga JavaScript-bibliotek eller moduler. Till exempel genom att använda:

```
npm install --save-dev @types/lodash
```

Installeras typdefinitionerna för `lodash` i ditt nuvarande projekt.

För att bidra till typdefinitionerna för @types-paket, skicka in en pull request till https://github.com/DefinitelyTyped/DefinitelyTyped.

JSX

JSX (JavaScript XML) är en utökning av JavaScript-språksyntaxen som låter dig skriva HTML-liknande kod i dina JavaScript- eller TypeScript-filer. Det används vanligtvis i React för att definiera HTML-strukturen.

TypeScript utökar JSX:s kapacitet genom att tillhandahålla typkontroll och statisk analys.

För att använda JSX behöver du ställa in kompileringsalternativet `jsx` i din `tsconfig.json`-fil. Två vanliga konfigurationsalternativ:

- "preserve": genererar .jsx-filer med JSX oförändrad. Detta alternativ talar om för TypeScript att behålla JSX-syntaxen som den är och inte transformera den under kompileringsprocessen. Du kan använda detta alternativ om du har ett separat verktyg, som Babel, som hanterar transformationen.
- "react": aktiverar TypeScripts inbyggda JSX-transformation. React.createElement kommer att användas.

Alla alternativ finns tillgängliga här: https://www.typescriptlang.org/tsconfig#jsx

ES6-moduler

TypeScript stöder ES6 (ECMAScript 2015) och många efterföljande versioner. Det innebär att du kan använda ES6-syntax, såsom pilfunktioner, mallsträngar, klasser, moduler, destrukturering och mer.

För att aktivera ES6-funktioner i ditt projekt kan du ange egenskapen `target` i tsconfig.json.

Ett konfigurationsexempel:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

ES7 exponentiationsoperator

Exponentiationsoperatorn (`**`) beräknar värdet som erhålls genom att upphöja den första operanden till den andra operandens potens. Den fungerar på liknande sätt som `Math.pow()`, men med den ytterligare förmågan att acceptera BigInts som operander. TypeScript stöder denna operator fullt ut genom att använda `es2016` eller en senare version som `target` i din tsconfig.json-fil.

```
console.log(2 ** (2 ** 2)); // 16
```

for-await-of-satsen

Detta är en JavaScript-funktion som stöds fullt ut i TypeScript och som låter dig iterera över asynkrona iterbara objekt från målversion es2018.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

Metaegenskapen new target

Du kan i TypeScript använda metaegenskapen `new.target` som gör det möjligt att avgöra om en funktion eller konstruktor anropades med new-operatorn. Det låter dig upptäcka om ett objekt skapades som ett resultat av ett konstruktoranrop.

```
class Parent {
    constructor() {
                    console.log(new.target);   // Skriver ut
konstruktorfunktionen som används för att skapa en instans
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}
```

```
const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

Dynamiska importuttryck

Det är möjligt att villkorligt ladda moduler eller lat-ladda dem vid behov med hjälp av ECMAScript-förslaget för dynamisk import som stöds i TypeScript.

Syntaxen för dynamiska importuttryck i TypeScript är följande:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
          const widget = await import('./widget'); // Dynamisk
import
        widget.render(container);
    }
}

renderWidget();
```

“tsc –watch”

Detta kommando startar TypeScript-kompilatorn med parametern `--watch`, med möjligheten att automatiskt kompilera om TypeScript-filer när de ändras.

```
tsc --watch
```

Från och med TypeScript version 4.9 förlitar sig filövervakning främst på filsystemhändelser och faller automatiskt tillbaka till polling om en händelsebaserad övervakare inte kan upprättas.

Non-null Assertion Operator

Non-null Assertion Operator (Postfix !) även kallad Definite Assignment Assertions är en TypeScript-funktion som låter dig försäkra att en variabel eller egenskap inte är null eller undefined, även om TypeScripts statiska typanalys antyder att den kan vara det. Med denna funktion är det möjligt att ta bort all explicit kontroll.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

Standarddeklarationer

Standarddeklarationer används när en variabel eller parameter tilldelas ett standardvärde. Det innebär att om inget värde anges för den variabeln eller parametern, kommer standardvärdet att användas istället.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

Valfri kedjning (Optional Chaining)

Den valfria kedjningsoperatorn `?.` fungerar som den vanliga punktoperatorn (`.`) för att komma åt egenskaper eller metoder. Den hanterar dock null- eller undefined-värden smidigt genom att avsluta uttrycket och returnera `undefined`, istället för att kasta ett fel.

```ts
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

Nullish coalescing-operatorn

Nullish coalescing-operatorn `??` returnerar högerledet om vänsterledet är `null` eller `undefined`; annars returnerar den vänsterledets värde.

```ts
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

Mallsträngslitteraltyper (Template Literal Types)

Mallsträngslitteraltyper gör det möjligt att manipulera strängvärden på typnivå och generera nya strängtyper baserade på befintliga. De är användbara för att skapa mer uttrycksfulla och precisa typer från strängbaserade operationer.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

Funktionsöverlagring

Funktionsöverlagring låter dig definiera flera funktionssignaturer för samma funktionsnamn, var och en med olika parametertyper och returtyp. När du anropar en överlagrad funktion använder TypeScript de angivna argumenten för att avgöra rätt funktionssignatur:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

Rekursiva typer

En rekursiv typ är en typ som kan referera till sig själv. Detta är användbart för att definiera datastrukturer som har en hierarkisk eller rekursiv struktur (potentiellt oändlig nästning), såsom länkade listor, träd och grafer.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

Rekursiva villkorstyper

Det är möjligt att definiera komplexa typrelationer med hjälp av logik och rekursion i TypeScript. Låt oss bryta ner det i enkla termer:

Villkorstyper: låter dig definiera typer baserade på booleska villkor:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

Rekursion: innebär en typdefinition som refererar till sig själv inom sin egen definition:

```
type Json = string | number | boolean | null | Json[] | { [key: string]: Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
        prop4: [],
    },
};
```

Rekursiva villkorstyper kombinerar både villkorslogik och rekursion. Det innebär att en typdefinition kan bero på sig själv genom villkorslogik, vilket skapar komplexa och flexibla typrelationer.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 | 1 | 6
```

Stöd för ECMAScript-moduler i Node

Node.js lade till stöd för ECMAScript-moduler från och med version 15.3.0, och TypeScript har haft stöd för ECMAScript-moduler i Node.js sedan version 4.7. Detta stöd kan aktiveras genom att använda egenskapen `module` med värdet `nodenext` i tsconfig.json-filen. Här är ett exempel:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js stöder två filändelser för moduler: `.mjs` för ES-moduler och `.cjs` för CommonJS-moduler. Motsvarande filändelser i TypeScript är `.mts` för ES-moduler och `.cts` för CommonJS-moduler. När TypeScript-kompilatorn transpilerar dessa filer till JavaScript skapar den `.mjs`- och `.cjs`-filer.

Om du vill använda ES-moduler i ditt projekt kan du ställa in egenskapen `type` till “module” i din package.json-fil. Detta instruerar Node.js att behandla projektet som ett ES-modulprojekt.

Dessutom stöder TypeScript även typdeklarationer i .d.ts-filer. Dessa deklarationsfiler tillhandahåller typinformation för

bibliotek eller moduler skrivna i TypeScript, vilket låter andra utvecklare använda dem med TypeScripts typkontroll och autokompletteringsfunktioner.

Assertionsfunktioner

I TypeScript är assertionsfunktioner funktioner som indikerar verifieringen av ett specifikt villkor baserat på sitt returvärde. I sin enklaste form undersöker en assert-funktion ett givet predikat och kastar ett fel när predikatet utvärderas till false.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

De kan också deklareras som funktionsuttryck:

```
type AssertIsNumber = (value: unknown) => asserts value is
number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

Assertionsfunktioner delar likheter med typskydd (type guards). Typskydd introducerades ursprungligen för att utföra körtidskontroller och säkerställa typen av ett värde inom ett specifikt scope. Specifikt är ett typskydd en funktion som utvärderar ett typpredikat och returnerar ett booleskt värde som anger om predikatet är sant eller falskt. Detta skiljer sig något från assertionsfunktioner, där avsikten är att kasta ett fel snarare än att returnera false när predikatet inte uppfylls.

Exempel på typskydd:

```
const isNumber = (value: unknown): value is number => typeof
value === 'number';
```

Variadiska tuppeltyper

Variadiska tuppeltyper är en funktion som introducerades i TypeScript version 4.0. Låt oss börja med att repetera vad en tuppel är:

En tuppeltyp är en array som har en definierad längd, och där typen av varje element är känd:

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

Termen "variadisk" betyder obestämd aritet (accepterar ett variabelt antal argument).

En variadisk tuppel är en tuppeltyp som har alla egenskaper som ovan men vars exakta form ännu inte är definierad:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

I den föregående koden kan vi se att tuppelns form definieras av den generiska typen `T` som skickas in.

Variadiska tuppler kan acceptera flera generiska typer vilket gör dem mycket flexibla:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

Med de nya variadiska tupplerna kan vi använda:

- Spridningar i tuppeltypssyntax kan nu vara generiska, så vi kan representera högre ordningens operationer på tuppler och arrayer även när vi inte känner till de faktiska typerna vi opererar på.
- Restelement kan förekomma var som helst i en tuppel.

Exempel:

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

Inkapslingstyper (Boxed types)

Inkapslingstyper refererar till omslagsobjekten som används för att representera primitiva typer som objekt. Dessa omslagsobjekt tillhandahåller ytterligare funktionalitet och metoder som inte är direkt tillgängliga på de primitiva värdena.

När du anropar en metod som `charAt` eller `normalize` på en primitiv `string`, omsluter JavaScript den i ett `String`-objekt, anropar metoden och kastar sedan bort objektet.

Demonstration:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript representerar denna skillnad genom att tillhandahålla separata typer för primitiverna och deras motsvarande objektomslag:

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

Inkapslingstyperna behövs vanligtvis inte. Undvik att använda inkapslingstyper och använd istället typer för primitiverna, till exempel `string` istället för `String`.

Kovarians och kontravarians i TypeScript

Kovarians och kontravarians beskriver relationer mellan typer i generiska typer. En `Dog` är en `Animal`, men inte varje `Animal` är en `Dog`.

Kovarians (producera värden): en funktion som returnerar en `Dog` kan ersätta en funktion som returnerar en `Animal`. Varje

hund är ett djur, men en funktion som returnerar vilket djur som helst kan returnera en katt.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;

produceAnimal = produceDog; // Giltigt
produceDog = produceAnimal; // Fel
```

Kontravarians (ta emot värden): en funktion som accepterar en valfri `Animal` kan ersätta en funktion som accepterar en `Dog`. Den kan också hantera hundar; en funktion för enbart hundar kan inte säkert hantera alla djur.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // Giltigt
consumeAnimal = consumeDog; // Fel med strictFunctionTypes
```

Pilarna visar riktningen för säkra tilldelningar:

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)

- Consumer<Animal> -> Consumer<Dog> (contravariance)

Föränderliga arrayer: TypeScript tillåter även att `Dog[]` tilldelas `Animal[]`. Båda variablerna pekar på samma array; via den bredare typen kan man lägga till en `Animal` som inte är en hund.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Tillåtet, men osäkert
// dogs innehåller nu ett Animal som inte är en Dog.
```

Kompilatorinställning: med `strictFunctionTypes` kontrolleras parametrar i fristående funktioner kontravariant. Utan inställningen kan TypeScript godta båda riktningarna (bivarians), även när det är osäkert. Parametrar i metod- och konstruktordeklarationer är undantag.

Valfria variansannotationer för typparametrar

Sedan TypeScript 4.7 betyder `out` kovarians, `in` kontravarians och `in out` invarians. TypeScript härleder normalt variansen. Annoteringarna måste stämma med hur den generiska parametern används och ändrar inte dess beteende godtyckligt.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

Mallsträngsmönsterindexsignaturer

Mallsträngsmönsterindexsignaturer gör det möjligt att definiera flexibla indexsignaturer med hjälp av

mallsträngsmönster. Denna funktion gör det möjligt att skapa objekt som kan indexeras med specifika mönster av strängnycklar, vilket ger mer kontroll och specificitet vid åtkomst och manipulering av egenskaper.

TypeScript tillåter från version 4.4 indexsignaturer för symboler och mallsträngsmönster.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Unik symbolnyckel
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

satisfies-operatorn

Operatorn `satisfies` låter dig kontrollera om en given typ uppfyller ett specifikt gränssnitt eller villkor. Med andra ord säkerställer den att en typ har alla nödvändiga egenskaper och metoder för ett specifikt gränssnitt. Det är ett sätt att säkerställa att en variabel passar in i en typdefinition. Här är ett exempel:

```ts
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Typannotering med `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// På följande rader kan TypeScript inte härleda typerna korrekt
user.attributes?.map(console.log); // Egenskapen 'map' finns inte på typen 'string | string[]'. Egenskapen 'map' finns inte på typen 'string'.
user.nickName; // string | string[] | undefined

// Typförsäkran med `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// Även här kan TypeScript inte härleda typerna korrekt
user2.attributes?.map(console.log); // Egenskapen 'map' finns inte på typen 'string | string[]'. Egenskapen 'map' finns inte på typen 'string'.
user2.nickName; // string | string[] | undefined

// Med operatorn `satisfies` kan vi nu härleda typerna korrekt
const user3 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} satisfies User;
```

```
user3.attributes?.map(console.log);   // TypeScript härleder
korrekt: string[]
user3.nickName; // TypeScript härleder korrekt: undefined
```

Importer och exporter av enbart typer

Importer och exporter av enbart typer låter dig importera eller exportera typer utan att importera eller exportera de värden eller funktioner som är associerade med dessa typer. Detta kan vara användbart för att minska storleken på ditt paket.

För att använda importer av enbart typer kan du använda nyckelordet `import type`.

TypeScript tillåter användning av både deklarations- och implementationsfiländelser (.ts, .mts, .cts och .tsx) i importer av enbart typer, oavsett inställningarna för `allowImportingTsExtensions`.

Till exempel:

```
import type { House } from './house.ts';
```

Följande former stöds:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

using-deklaration och explicit resurshantering

En `using`-deklaration är en blockomfattande, oföränderlig bindning, liknande `const`, som används för att hantera disponibla resurser. När den initialiseras med ett värde

registreras värdets `Symbol.dispose`-metod och körs sedan vid utgång ur det omslutande blockomfånget.

Detta baseras på ECMAScripts resurshanteringsfunktion, som är användbar för att utföra väsentliga uppstädningsuppgifter efter objektskapande, såsom att stänga anslutningar, radera filer och frigöra minne.

Noteringar:

- På grund av dess nyliga introduktion i TypeScript version 5.2 saknar de flesta körmiljöer inbyggt stöd. Du behöver polyfills för: `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack`, `SuppressedError`.
- Dessutom behöver du konfigurera din tsconfig.json enligt följande:

```
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

Exempel:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Enkel polyfill

const doWork = (): Disposable => {
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};
```

```
console.log(1);

{
    using work = doWork(); // Resursen deklareras
    console.log(2);
} // Resursen frigörs (t.ex. utvärderas `work[Symbol.dispose]
()`)

console.log(3);
```

Koden loggar:

```
1
2
disposed
3
```

En resurs som kan disponeras måste följa `Disposable`-gränssnittet:

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

`using`-deklarationer registrerar resursavyttringsoperationer i en stack, vilket säkerställer att de disponeras i omvänd ordning mot deklarationen:

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // frigör `C`, sedan `B`, sedan `A`.
```

Resurser garanteras att disponeras, även om efterföljande kod eller undantag uppstår. Detta kan leda till att avyttringen potentiellt kastar ett undantag, som eventuellt undertrycker ett annat. För att behålla information om undertryckta fel introduceras ett nytt inbyggt undantag, `SuppressedError`.

await using-deklaration

En `await using`-deklaration hanterar en asynkront disponibel resurs. Värdet måste ha en `Symbol.asyncDispose`-metod, som kommer att inväntas vid blockets slut.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // Resursen deklareras
}    //    Resursen    frigörs    (t.ex.    utvärderas    `await
work[Symbol.asyncDispose]()`)
```

För en asynkront disponibel resurs måste den följa antingen `Disposable`- eller `AsyncDisposable`-gränssnittet:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose ??= Symbol('Symbol.asyncDispose'); // Enkel
polyfill

class DatabaseConnection implements AsyncDisposable {
    // En metod som anropas när objektet frigörs asynkront
    [Symbol.asyncDispose]() {
        // Stäng anslutningen och returnera ett löfte
        return this.close();
    }
```

```
    async close() {
        console.log('Closing the connection...');
            await new Promise(resolve => setTimeout(resolve,
1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
    // Skapa en ny anslutning och frigör den asynkront när den
lämnar sitt räckviddsområde
      await using connection = new DatabaseConnection(); //
Resursen deklareras
    console.log('Doing some work...');
}   //   Resursen   frigörs   (t.ex.   utvärderas   `await
connection[Symbol.asyncDispose]()`)

doWork();
```

Koden loggar:

```
Doing some work...
Closing the connection...
Connection closed.
```

Deklarationerna `using` och `await using` är tillåtna i följande satser: `for`, `for-in`, `for-of`, `for-await-of`, `switch`.

Importattribut

TypeScript 5.3:s importattribut (etiketter för importer) talar om för körmiljön hur moduler (JSON, etc.) ska hanteras. Detta förbättrar säkerheten genom att säkerställa tydliga importer och överensstämmer med Content Security Policy (CSP) för säkrare resursladdning. TypeScript säkerställer att de är giltiga men låter körmiljön hantera deras tolkning för specifik modulhantering.

Exempel:

```
import config from './config.json' with { type: 'json' };
```

med dynamisk import:

```
const config = import('./config.json', { with: { type: 'json' }
});
```

Syntaxkontroll för reguljära uttryck

Sedan TypeScript 5.5.4 kontrollerar den regex-literaler för vanliga fel vid kompileringstid (t.ex. ogiltig syntax, felaktiga bakåtreferenser, funktioner som inte stöds för din mål-JS-version). Den hjälper till att upptäcka buggar tidigare, men kontrollerar inte nya RegExp(“…)-strängar.

```
let r = /(a)\2/; // Fel: Denna bakåtreferens refererar till en
grupp som inte finns.
```

import defer

`import defer` låter dig ladda en modul men fördröja dess körning tills du faktiskt använder något från den. Detta hjälper till att undvika onödigt arbete och biverkningar.

- Fungerar bara med: `import defer * as name from "module"`
- Koden körs bara när du öppnar en export